建立Zynq工程

2026/10/27 12:36:58 · 50 条消息（用户 25 · ChatGPT 25） · 原始会话

> 16 张图片无法内嵌，已保留原始链接；离线或链接过期后可能无法显示。

你

好的，让我们开始下一步，建立zynq工程，开展板级测试

ChatGPT

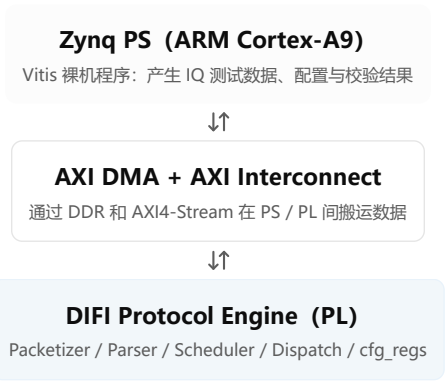
我们现在正式进入 **DIFI 1.1 FPGA Digital IF 协议引擎的 Zynq 板级验证阶段。**

我建议沿用之前确定的原则：**先验证硬件数据通路，再验证完整协议引擎，最后接入 UDP 和 PC 端的 DIFI 标准验证。**

目前已经核对到你的 GitHub 开发分支 `feature/rtl-integration` 中的 `rtl/top/difi_engine_top.v`。它是完整九模块协议引擎的结构顶层，但还没有包含 AXI DMA、时间源和板级控制逻辑。因此，我们需要在它外面建立 Zynq 系统，而不是直接把它作为 FPGA 引脚顶层。

一、我们要实现什么？

第一阶段目标：Zynq PS ↔ PL 真实数据收发



这是目标系统的逻辑架构图；实际硬件将分别使用 DMA 的 MM2S 和 S2MM 通道，且需要处理 DIFI Packet 边界。

我们按三个层次逐步验证：

测试	具体内容	通过标准
A：基础 TX 上板	PS IQ → Data Packetizer → PS Packet	PS 收到正确的 DIFI 包
B：完整引擎上板	Data / Context / Version 发送和解析	三类包、配置和 IQ 正确
C：外部通信闭环	PC ↔ UDP ↔ PS ↔ DIFI PL	Wireshark 抓包、外部工

今天先完成 A 的硬件工程搭建准备。 这能将 DMA、时钟、复位和协议 RTL 的问题分开排查，避免第一次上板就同时调试整个系统。

二、创建 Vivado 工程

硬件平台先使用你之前做过图像实验的开发板。

项目	本次配置
开发板	正点原子领航者 ZYNQ-7020
FPGA 器件	`xc7z020clg400-2`
Vivado	2024.2
Vitis	2024.2

PL 时钟	100 MHz
PS 软件	Bare-metal 裸机
DMA 工作方式	Simple Mode，先使用轮询
数据宽度	32 bit

芯片型号也与领航者 7020 的官方教程一致。

m.ruidan.com

第一步：准备本次板级测试 RTL

我已根据当前 GitHub 仓库里的 `difi_data_packetizer.v` 接口，编写了一个第一阶段测试用的板级适配模块。

下载 difi_tx_board_test.v ↗

Stage A 专用测试 Wrapper，Verilog-2001

这个 Wrapper 做了三件事：将 32-bit AXI4-Stream 接口引出供 DMA 连接；把 Stream ID 固定为 `1`、样本数固定为 `256`；通过 AXI GPIO 的输出电平上升沿产生一次 `pkt_start`。

时间戳暂时使用固定测试值，不代表真实 UTC 时间。正式版本会改成有效时间源。

把它放到你的 GitHub 工作目录下，例如：

纯文本

```
DIFI/
├── rtl/
│   ├── include/
│   │   └── difi_v1_1_defs.vh
│   ├── data/
│   │   └── difi_data_packetizer.v
│   └── board/
│       └── difi_tx_board_test.v    ← 新增
└── prj/
    └── vivado/
```

第二步：创建空工程

1. 打开 Vivado 2024.2，选择 `Create Project`。
2. 工程名填写 `difi_zynq_stage_a`，保存到 `D:/Study/project/DIFI/prj/vivado/`。
3. 选择 `RTL Project`，可暂时勾选 `Do not specify sources at this time`。
4. 在 `Parts` 中选择 `xc7z020clg400-2`，点击 `Finish`。
5. 通过 `Add Sources → Add or Create Design Sources`，添加刚才的 Wrapper、`difi_data_packetizer.v` 和协议头文件。

由于 Packetizer 使用了：

verilog

```
`include "difi_v1_1_defs.vh"
```

还要在 Vivado 的 Tcl Console 中配置头文件搜索路径：

tcl

```
set_property include_dirs \
  [list "D:/Study/project/DIFI/rtl/include"] \
  [get_filesets sources_1]
```

实际路径按你的本地仓库位置调整。

三、建立 Block Design

第三步：添加 Zynq PS

进入 `IP INTEGRATOR → Create Block Design`，命名为 `difi_system`。

通过 `Add IP` 添加 `ZYNQ7 Processing System`，然后配置：

PS 参数	建议配置
M_AXI_GP0	Enable
S_AXI_HP0	Enable
FCLK_CLK0	100 MHz
FCLK_RESET0_N	Enable
DDR、MIO、UART	使用开发板的原有配置

特别注意 DDR 与 MIO 设置。 最好复用你之前已经成功运行摄像头、VDMA 的工程中的 PS 配置，或者导入正点原子的板级配置，不要直接把默认 PS 参数当作开发板实际配置。

第四步：添加其余 IP

在 Block Design 中添加下列 IP：

IP	数量	作用
AXI Direct Memory Access	1	IQ 与 DIFI Packet 的 DDR 数据搬运
AXI GPIO	1	PS 控制 Packetizer 开始发包
Processor System Reset	1	统一复位
AXI SmartConnect / Interconnect	按连接自动生成	AXI 总线互联
`difi_tx_board_test`	1	刚才编写的 RTL

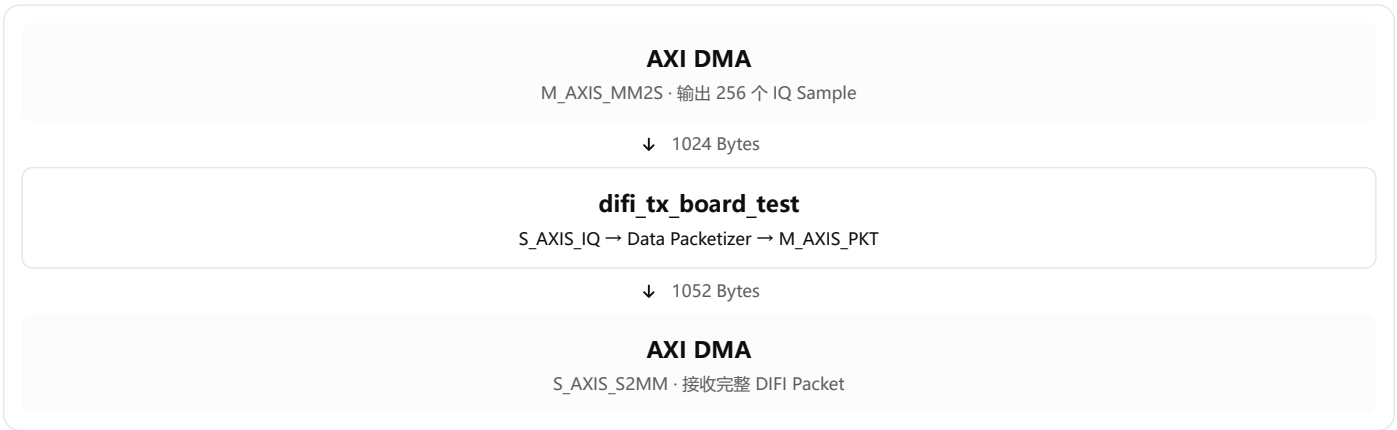
配置 AXI DMA 时，取消 `Enable Scatter Gather Engine`，同时开启 MM2S 和 S2MM 通道，并采用 32-bit Stream 数据宽度。

Simple Mode 的 DMA 可以由 PS 软件设置缓冲区地址及传输长度来启动，比较适合第一轮板级调试。

PG021

第五步：连接数据通路

核心 AXI4-Stream 连接如下：



其余连接中，PS 的 `M_AXI_GP0` 经 AXI 互联访问 DMA 控制寄存器和 AXI GPIO；DMA 的两个存储器映射主接口经 AXI 互联接入 PS 的 `S_AXI_HP0`，用于访问 DDR。

所有有关 PL 的 AXI 时钟和 DIFI 的 `aclk` 使用同一个 100 MHz 时钟，低有效复位接到 `proc_sys_reset` 的 `peripheral_aresetn`。GPIO 配为单通道、1-bit、全输出，连接 `start_level`。

Wrapper 的 `pkt_busy`、`pkt_done` 可在后续通过 ILA 观察，第一版不必连接到 PS。

完成后运行 `Validate Design`，再生成 HDL Wrapper，设置生成的 BD Wrapper 为 Vivado Top。

四、第一轮板级验证应该看到什么?

我们先选择 256 个 IQ Sample 作为输入。

DMA MM2S 输入	DMA S2MM 接收
1024 B	**1052 B**
256 × 4 Bytes	(7 + 256) × 4 Bytes

Vitis 程序后续按照以下顺序工作：准备 IQ 缓冲区，配置并启动 S2MM 接收，利用 GPIO 产生一次启动脉冲，再启动 MM2S 发送 IQ，等待两路 DMA 完成，最后对接收缓冲区进行 Cache Invalidate 并逐 Word 比对。

第一包的期望结果为：

Word	预期内容
Word0	`0x18600107`
Word1	`0x00000001`
Word2	`0x006A621E`
Word3	`0x00000000`
Word4	`0x6553F100`（固定测试时间）
Word5–6	`0x00000000`
Word7–262	与原始 256 个 IQ Sample 完全一致

PS 以 32-bit 整数比较时无需交换字节序；未来把这些数据通过 UDP 发送时，则必须按协议要求转换成网络大端字节序。

五、我们当前就做这一件事

先把 **Vivado Block Design 创建好，并正确连接 Zynq PS、DMA、AXI GPIO 和** `difi_tx_board_test`。

不需要马上写 Vitis 程序，也不用现在接入 Context、Version 或完整 `difi_engine_top`。

完成后把你的 **Vivado Block Design 截图**发给我。我会先帮你逐项检查 AXI 总线、时钟、复位与 DMA 数据流连接，然后指导你进行综合、生成 Bitstream、导出 XSA，并编写第一份 Vitis 裸机测试程序。

我们的第一个明确验收条件就是：

PS 写入 1024 Bytes IQ 数据，经过真正的 FPGA Packetizer，PS 从 DDR 读回 1052 Bytes 完全符合预期的 DIFI Signal Data Packet。

你



这一页怎么设置？以及连接数据通路时如何把刚才你写的rtl添加到block design供连线?

ChatGPT

你这张截图中的 AXI DMA 配置**基本已经正确了，不需要进行大的修改。**我们现在使用的是 Zynq-7020 + AXI DMA + DIFI Packetizer 的第一阶段板级测试方案，重点是先验证数据能否正确从 DDR 经过 FPGA 协议模块，再写回 DDR。

我按照你现在的 Vivado 2024.2 界面，一步一步说明。

一、AXI DMA 这一页如何设置?

✓ 你截图中的配置可以直接使用

核对如下参数即可：

配置项	应设置为
Enable Scatter Gather Engine	不勾选
Enable Micro DMA	不勾选
Width of Buffer Length Register	14 bits
Address Width	32 bits
Enable Read Channel	勾选
Enable Write Channel	勾选
Memory Map Data Width	32 bits
Stream Data Width	32 bits
Max Burst Size	16
Allow Unaligned Transfers	不勾选

这里解释三个比较重要的参数。

1. Width of Buffer Length Register = 14

它决定了 DMA 单次传输长度寄存器能够表示的最大字节数。

14 bits 允许最大设置 16383 Bytes，而我们第一轮测试：

- MM2S：1024 Bytes
- S2MM：1052 Bytes

所以完全足够，不必改成 23 或 26 bits。

2. Memory Map Data Width 和 Stream Data Width

二者都设为 32 bits。

因为我们 DIFI RTL 的 IQ 输入与 Packet 输出都采用 32-bit AXI4-Stream，每个时钟周期传送一个 32-bit Word。

3. Allow Unaligned Transfers 不勾选

我们暂时不需要 DRE（Data Realignment Engine）。Vitis 后续分配和使用的 DMA 缓冲区地址确保至少 4-byte 对齐即可。

因此，这一页你可以直接点击右下角的 **OK**。

二、如何将自己的 RTL 添加到 Block Design?

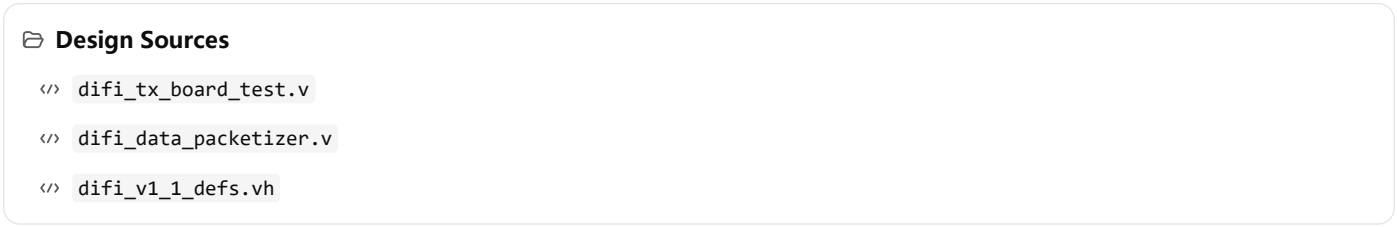
这里有一个 Vivado 非常有用的功能，叫作 **Module Reference**。

它可以把普通 Verilog 模块直接添加到 Block Design 中，无须先封装成自定义 IP。

我们这一步就使用它。

第一步：检查 Design Sources

在 Vivado 左侧的 `Sources → Design Sources` 中，确保已经加入这三个文件：



其中：

- `difi_tx_board_test.v`：我们要放进 Block Design 的测试 Wrapper。
- `difi_data_packetizer.v`：真正执行 DIFI 封包的 RTL 模块。
- `difi_v1_1_defs.vh`：协议常量定义。

如果还没添加，点击：

`Add Sources → Add or Create Design Sources → Add Files`

选择文件，完成添加。

注意，我们应该把 `difi_tx_board_test` 放入 Block Design，而不是直接把 `difi_data_packetizer` 放进去。前者已经实例化了后者，并且提供了便于连接 DMA 的完整 AXI4-Stream 接口。

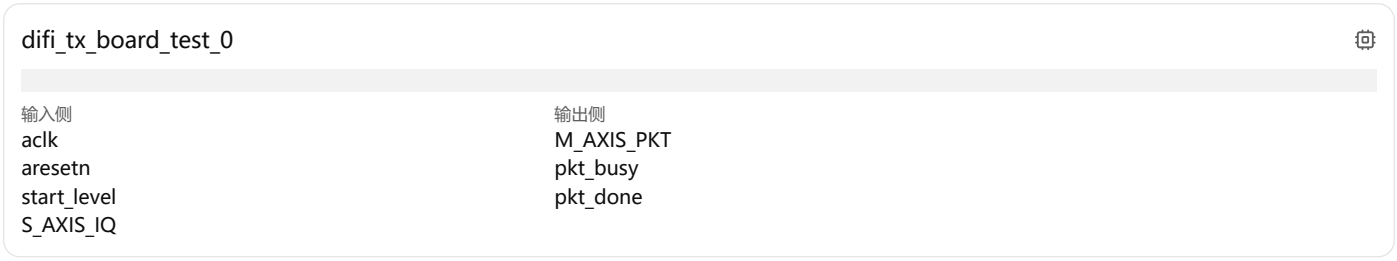
第二步：在 Block Design 中添加 Module

打开你创建的 `difi_system` Block Design。

按以下操作：

1. 在 Block Design 的空白区域点击右键。
2. 选择 **Add Module**（不是 Add IP）。
3. 搜索或选择 `difi_tx_board_test`。
4. 点击确认，将模块添加到 Block Design。

成功之后，会出现一个新的模块方框，类似下面这样：



我之前写的 Wrapper 已经包含 `X_INTERFACE_INFO` 属性，因此 Vivado 应当能够自动识别 `S_AXIS_IQ` 和 `M_AXIS_PKT` 两个 AXI4-Stream 接口，将 TDATA、TVALID、TREADY、TKEEP 和 TLAST 组合成总线连接口。

如果没有自动组合，也可以使用单独信号连接，我们到时根据界面处理。

第三步：连接 AXI DMA 与 RTL

现在找到 AXI DMA 模块上的三个接口：

AXI DMA 端口	连接到
`M_AXIS_MM2S`	`difi_tx_board_test_0/S_AXIS_IQ`
`S_AXIS_S2MM`	`difi_tx_board_test_0/M_AXIS_PKT`
`S_AXI_LITE`	PS `M_AXI_GP0` 经 AXI Interconnect

前两条是我们的核心数据通路。

其中：

- `M_AXIS_MM2S` 是 DMA 向 PL 输出 IQ 的接口。
- `S_AXIS_S2MM` 是 DMA 从 PL 接收 DIFI Packet 的接口。

连接时只需要拖动对应的 AXI4-Stream 总线端口，无需自己逐根连 TDATA、TVALID、TREADY。

第四步：连接时钟、复位以及 GPIO

在第一轮中使用统一 100 MHz 时钟。

信号	连接
`difi_tx_board_test_0/aclk`	`FCLK_CLK0`
`difi_tx_board_test_0/aresetn`	`proc_sys_reset/peripheral_aresetn`
`difi_tx_board_test_0/start_level`	AXI GPIO 的 `gpio_io_o[0]`
`axi_dma_0/s_axi_lite_aclk`	`FCLK_CLK0`
`axi_dma_0/m_axi_mm2s_aclk`	`FCLK_CLK0`
`axi_dma_0/m_axi_s2mm_aclk`	`FCLK_CLK0`
`axi_dma_0/axi_resetn`	`peripheral_aresetn`

对于 AXI DMA 访问 DDR 的部分，还需要连接：

- `M_AXI_MM2S` → 经 AXI Interconnect → PS `S_AXI_HP0`
- `M_AXI_S2MM` → 经 AXI Interconnect → PS `S_AXI_HP0`

PS 的 `M_AXI_GP0_ACLK`、`S_AXI_HP0_ACLK` 也要连接到相应的 100 MHz 时钟。

这些 AXI Interconnect 和 SmartConnect，可以优先使用 Vivado 的 **Run Connection Automation** 自动创建和连接，不必全部手工搭建。

另外，AXI GPIO 配置为单通道、宽度 1 bit、全输出，初始化值为 0。

三、完成之后，我们要检查什么？

建议你按照下面的清单完成本阶段。

板级工程配置检查 0/9 项

- [ ] AXI DMA 参数按截图配置完毕
- [ ] 将三个 RTL 文件加入 Design Sources
- [ ] 使用 Add Module 添加 difi_tx_board_test
- [ ] M_AXIS_MM2S 与 S_AXIS_IQ 连接
- [ ] M_AXIS_PKT 与 S_AXIS_S2MM 连接
- [ ] AXI GPIO 连接 start_level
- [ ] PS GP0、HP0 与 DMA 控制/存储器接口连接
- [ ] 100 MHz 时钟和复位全部连接
- [ ] 点击 Validate Design，无 Critical Error

现在暂时**不要点击 Generate Bitstream。**先确认 Block Design 的结构正确，特别是 DMA 的存储器映射接口、Stream 接口和时钟。

另外，由于这一轮不是把原来的 `difi_engine_top` 接入板子，我们暂时不需要 AXI4-Lite 配置 DIFI 协议模块。GPIO 启动信号和固定配置只服务于第一轮最小验证。

你下一步就先点击 DMA 配置的 OK，然后使用 Add Module 把 `difi_tx_board_test` 添加到 Block Design。

如果添加成功，给我发一张整个 Block Design 的截图，我们继续逐根核对连接，避免后面出现 DMA 卡死、TLAST 不正确或者数据收不到的问题。

你



这样算是连好了吗?

ChatGPT

我仔细检查了你的 Block Design。**整体架构已经基本搭建正确，但还不能算完全连好。** 目前最明显的是 **AXI GPIO 没有连接到** `start_level`，另外复位模块有两个输入需要确认。

好消息是，你连接的 DMA 数据通路、PS 访问 DDR 的路径以及 AXI 控制路径，从截图上看都基本正确，不需要推倒重来。

一、目前连接情况

检查项目	结果	说明
DMA `M_AXIS_MM2S` → DIFI `S_AXIS_IQ`	✅	已连接
DIFI `M_AXIS_PKT` → DMA `S_AXIS_S2MM`	✅	已连接
DMA `M_AXI_MM2S` → PS `S_AXI_HP0`	✅	已通过 Interconnect 连接
DMA `M_AXI_S2MM` → PS `S_AXI_HP0`	✅	已通过 Interconnect 连接
PS `M_AXI_GP0` → DMA、GPIO 控制接口	✅	SmartConnect 连接基本正确
时钟和主要复位线路	✅	看起来已连接，仍需验证
GPIO → `start_level`	❌	未连接，必须修改
`proc_sys_reset/dcm_locked`	⚠️	未看到连接，需要处理
`proc_sys_reset/aux_reset_in`	⚠️	未看到连接，建议明确设置

二、必须修改：GPIO 连接错误

你的截图右下角是：

axi_gpio_0 → `gpio_rtl_0`（外部端口）

当前：GPIO 被引出了 FPGA 顶层，但没有控制 DIFI 模块。

而我们的目标是：

axi_gpio_0 → `start_level` **DIFI**

目标：PS 通过 AXI GPIO 控制 Packetizer 启动。

具体修改步骤

1. 双击 `axi_gpio_0`，设置 GPIO Channel 1 的宽度为 `1`，方向为 `All Outputs`，默认输出值为 `0`，关闭不需要的双通道配置。
2. 点击 OK。
3. 右键删除右侧的外部接口 `gpio_rtl_0`，注意只删除外部端口，不要删除 AXI GPIO IP。

4. 展开 `axi_gpio_0` 右侧的 `GPIO` 接口（点击旁边的小 `+`）。
5. 找到 `gpio_io_o[0:0]` 输出端口。
6. 把它连接到 `difi_tx_board_test_0/start_level`。

这一步完成后，PS 软件就能够通过 AXI GPIO 输出 `0 → 1 → 0`，触发我们编写的 Packetizer 启动逻辑。

三、需要修改：Processor System Reset

在截图左下角，`proc_sys_reset_0` 有一个输入叫：

`dcm_locked`

目前看起来没有连接。

这个信号告诉复位控制器：外部时钟已经稳定，可以释放系统复位。

因为我们直接使用 Zynq PS 的 `FCLK_CLK0`，没有额外使用 Clocking Wizard，所以建议将 `dcm_locked` 明确接为逻辑 `1`。

具体操作：

1. `Add IP`，搜索 `Constant`。
2. 添加 `xlconstant`。
3. 配置 `Const Width = 1`，`Const Val = 1`。
4. 将它的 `dout` 连接到 `proc_sys_reset_0/dcm_locked`。
5. 同时可以将 `aux_reset_in` 接逻辑 `1`（未使用辅助复位时保持非有效状态）。

`ext_reset_in` 在截图里已经有连接，暂时不用修改。`mb_debug_sys_rst` 如果没有使用，可以保持为非有效值 `0`。

四、其余未连接端口需要处理吗？

不需要全部连接。

未连接端口	第一轮是否需要
`axi_dma_0/mm2s_introut`	不需要，Vitis 采用轮询模式
`axi_dma_0/s2mm_introut`	不需要
`difi_tx_board_test_0/pkt_busy`	可以暂时悬空
`difi_tx_board_test_0/pkt_done`	可以暂时悬空
`processing_system7_0/S_AXI_HP0_FIFO_CTRL`	不需要

这些暂时不连接不会妨碍第一轮 DMA 收发测试。

五、完成修改后，下一步做什么？

先不要急着综合，按照下面的顺序检查：

1. 点击 **Regenerate Layout**，整理连接图。
2. 点击 **Validate Design**，检查 IP 接口、时钟与复位连接。
3. 进入 **Address Editor**，确认 AXI DMA 和 AXI GPIO 都有有效的 PS 地址映射，且 DMA 的 DDR 地址空间可访问。
4. 保存 Block Design。

需要提醒：`Validate Design` 成功只能说明 Vivado 没发现明显的 Block Design 连接错误，并不能证明 DMA 已经能够成功收发数据。这个结论必须等 Vitis 板级测试后才能得出。

现在优先修改 GPIO → `start_level` 连接和复位模块的 `dcm_locked`。

改好后，先点击 `Validate Design`。如果验证通过，把提示结果或者更新后的 Block Design 截图发过来，我们就继续做 **Generate HDL Wrapper → Synthesis → Implementation → Generate Bitstream → Export XSA**。

你

验证通过，你说的我都连了，下一步干什么

ChatGPT

很好。既然 **Block Design 已经通过 Validate Design，且之前指出的连接问题都已经修正**，我们就进入下一阶段：

生成 HDL Wrapper → 综合（Synthesis）→ 布局布线（Implementation）→ 生成 Bitstream → 导出 XSA。

我也重新检查了之前写的 `difi_tx_board_test.v`。它使用 GPIO 上升沿产生 `pkt_start`，DIFI Packetizer 固定每包 256 个 IQ Sample，输出最后一个 Word 时产生 `TLAST`，适合当前的第一轮 DMA 板级测试。

一、生成 HDL Wrapper

这是你现在首先要做的事情。

Step 1：生成 Block Design 的输出文件

在 Vivado 左侧 `Sources` 窗口中找到：

`Design Sources → difi_system.bd`

右键点击 `difi_system.bd`，选择：

Generate Output Products

保持默认配置，点击 `Generate`，等待完成。如果此前已经自动生成，可跳过。

Step 2：创建 HDL Wrapper

再次右键点击 `difi_system.bd`：

选择 **Create HDL Wrapper。**

在弹出的窗口中选择：

> **Let Vivado manage wrapper and auto-update**
> 让 Vivado 自动管理顶层文件。Block Design 后续改动时，Vivado 可以更新 Wrapper。

点击 OK。

完成后，Sources 应该类似：

```
纯文本
Design Sources
└── difi_system_wrapper (difi_system_wrapper.v)
    └── difi_system_i (difi_system.bd)
        ├── processing_system7_0
        ├── axi_dma_0
        ├── axi_gpio_0
        ├── difi_tx_board_test_0
        └── ...
```

确认 `difi_system_wrapper` 是顶层模块。如果不是，右键它并选择 `Set as Top`。

注意： `difi_tx_board_test.v` 虽然叫 Wrapper，但它只是 DIFI 模块的板级适配层；整个 FPGA 的真正顶层应当是 `difi_system_wrapper.v`。

二、运行综合（Synthesis）

Step 3：Run Synthesis

在 Vivado 左侧 Flow Navigator 中点击：

`SYNTHESIS → Run Synthesis`

保持默认配置，点击 OK。

Vivado 会把 Verilog RTL 和 Block Design 中的 IP 一起综合成 FPGA 网表。

综合完成后，选择 `Open Synthesized Design`。

重点检查以下内容：

检查项	期望结果
Synthesis	Completed
Errors	0
DIFI 模块	成功综合
AXI DMA、GPIO、PS	成功加入网表
LUT / FF / BRAM	有实际资源使用报告

在 `Reports → Report Utilization` 中查看资源消耗。

这里需要区分：我们之前的 DIFI Engine OOC 资源报告对应的是完整协议核心，而本次是只有一个 Data Packetizer 的 Zynq 系统，所以两者的 LUT、FF、BRAM 不应该直接比较。

如果出现 Warning，先不要全部当作错误。比如 Packetizer 的输入 `TKEEP/TLAST` 在这个固定样本数测试中故意不使用，相关未使用信号提示不一定是问题。但若出现未连接时钟、复位、Black Box 或未约束时钟等问题，需要处理。

三、布局布线（Implementation）

Step 4：Run Implementation

综合完成后，点击：

`IMPLEMENTATION → Run Implementation`

保持默认策略。

Vivado 会执行布局、布线和时序优化。

完成后选择 `Open Implemented Design`。

Step 5：检查时序是否收敛

进入：

`Reports → Timing → Report Timing Summary`

主要看两个指标：

指标
WNS（Worst Negative Slack）
TNS（Total Negative Slack）

我们目前使用 **100 MHz PL 时钟**，对应周期为 10 ns。

如果 WNS 为正且没有未检查的关键时序路径，说明当前约束下的建立时间检查通过。

同时查看是否存在 Unconstrained Paths（未约束路径），不能仅凭 WNS 为正就认为全部时序正确。

四、生成 Bitstream

Step 6：Generate Bitstream

当 Implementation 完成且时序检查没有问题后，在 Flow Navigator 中点击：

PROGRAM AND DEBUG → Generate Bitstream

等待生成完成。

成功后，一般可以在工程目录下找到：

```
纯文本
difi_zynq_stage_a/
└── difi_zynq_stage_a.runs/
    └── impl_1/
        └── difi_system_wrapper.bit
```

具体文件路径取决于你创建 Vivado 工程时的目录和顶层名称。

`.bit` 是最终用于配置 FPGA PL 的文件。

五、导出硬件平台 XSA

Step 7：Export Hardware

Bitstream 生成成功后，在 Vivado 顶部选择：

`File → Export → Export Hardware`

如果弹出导出向导：

1. 选择包含 Bitstream 的硬件平台导出方式，即 **Include Bitstream**。
2. 选择保存路径。
3. 将文件命名为 `difi_zynq_stage_a.xsa`。
4. 完成导出。

建议将生成的 XSA 放在：

```
纯文本
D:/Study/project/DIFI/
└── board/
    └── zynq7020/
        └── export/
            └── difi_zynq_stage_a.xsa
```

XSA 文件中记录了 PS 配置、IP 地址映射、硬件接口等信息，并可包含生成的 Bitstream。

后续 Vitis 2024.2 会通过它创建硬件平台，让 ARM 端裸机程序访问 AXI DMA 和 AXI GPIO。

六、完成这一步后，我们接下来做什么？

板级验证进度

Block Design 连接与验证

已完成

综合、Implementation、Bitstream

当前任务

Vitis 裸机工程

创建 Zynq FSBL、Standalone Platform 和测试程序

AXI DMA 真实收发

1024 Bytes IQ 输入 → 1052 Bytes DIFI Packet 输出

逐 Word 数据校验

检查 Header、Stream ID、Timestamp、256 个 IQ Sample

你现在先完成上述 Vivado 流程，目标是成功生成 `.bit` 和 `.xsa` 文件。

完成之后，把 Vivado 的 **Implementation Timing Summary（时序报告）和 Utilization（资源报告）** 发给我。我会帮你确认 FPGA 是否真正通过了实现阶段的检查，然后我们开始 Vitis 2024.2 裸机工程，编写 AXI DMA 收发测试程序，完成第一次真实板级验证。

你



实现过后有好多timing严重警告

ChatGPT

你这个情况需要处理。**目前最值得关注的是 `no_clock (61)`，它比普通的时序裕量不足更值得优先排查。**

从截图看，Vivado 报告的是 `Check Timing` 检查结果，并不是 308 条路径全部发生了时序违例。

一、这些警告分别是什么意思?

警告	数量	含义
`no_clock`	61	部分时序端点没有被有效时钟约束覆盖
`unconstrained_internal_endpoints`	174 左右	内部存在未受完整时序约束的端点
`no_input_delay`	36	部分输入端口没有输入延迟约束
`no_output_delay`	37	部分输出端口没有输出延迟约束
`multiple_clock`	0	未发现相关多时钟问题
`latch_loops`	0	未发现相关锁存环路

尤其是截图右侧，你的 `difi_data_packetizer` 中出现了：

- `current_state_reg[0]/D`
- `current_state_reg[0]/CLR`
- `first_sample_reg[0]/D`
- `start_level_d_reg/D`

这些都是我们自己写的 DIFI RTL 内部的时序逻辑端点。

正常情况下，它们应该受到 100 MHz PL 时钟约束。

所以现在不能因为 Implementation 已经完成，就直接认定设计满足 100 MHz 时序。

二、最可能的问题：PL 时钟没有被正确识别或约束

根据目前的截图，我优先怀疑以下两种情况：

1. PS 的 `FCLK_CLK0` 虽然在 Block Design 中连接了 DIFI 模块，但 Vivado 没有为这一路时钟建立有效的时序约束。

2. PS 时钟本身有约束，但约束没有正确传播到 `difi_tx_board_test` 内部的寄存器。

我们需要先分辨是哪一种。

AMD 官方的 UG949 也明确建议先检查时钟定义及其覆盖情况，再判断实际时序是否收敛。

UG949 +1

第一步：检查 Vivado 识别到了哪些时钟

保持 `Open Implemented Design` 状态，在底部 **Tcl Console** 中输入：

```tcl
report_clocks
```

查看输出中是否存在与 `FCLK_CLK0` 对应的时钟。

正常应当看到类似：

```
Clock Name                    Period(ns)
-----------------------------------------
processing_system7_0_FCLK_CLK0    10.000
```

名称可能不同，关键是存在来自 Zynq PS 的 10 ns 时钟。

如果没有这一路时钟，就说明存在时钟约束缺失问题。

第二步：检查时钟网络

继续输入：

```tcl
report_clock_networks
```

重点寻找报告中的 `Unconstrained Clocks`。

如果这里出现 Zynq PS 的 `FCLKCLK[0]`，或者连接到 DIFI 模块的时钟源，就找到了重要线索。

第三步：单独检查 no_clock

输入：

```tcl
check_timing -verbose -override_defaults {no_clock}
```

这会单独列出缺少有效时钟约束的时钟管脚，比在 308 条检查结果中逐项查找更方便。

三、同时检查 Block Design 的时钟配置

回到 Block Design，双击 `processing_system7_0`：

进入 **Clock Configuration → PL Fabric Clocks**。

确认：

参数	期望
FCLK_CLK0	Enabled
Requested Frequency	100 MHz
实际配置频率	接近或等于 100 MHz

再确认 `FCLK_CLK0` 的网络连接到 `difi_tx_board_test_0/aclk`。

你也可以在打开 Block Design 时，在 Tcl Console 分别输入：

```tcl
get_bd_nets -of_objects [get_bd_pins processing_system7_0/FCLK_CLK0]
```

```tcl
get_bd_nets -of_objects [get_bd_pins difi_tx_board_test_0/aclk]
```

如果两者返回相同的 BD 网络，说明逻辑连线存在；如果不是同一网络，就需要修正连接。

但要注意：**连线正确不等于时序约束正确**，我们仍然需要前面生成的时钟报告。

四、其他警告暂时不用急着修

`no_input_delay (36)` 和 `no_output_delay (37)` 很可能来自 Zynq PS 的 DDR 或 FIXED_IO 顶层端口。

如果确实如此，它们与普通 PL 外接同步接口不同，通常不需要你手动添加输入、输出延迟约束。

现在不要为了消除警告而随意添加：

```tcl
set_false_path ...
```

也不建议直接在 DIFI 内部随便添加一个 `create_clock`。我们应该先确定 PS 时钟源及其约束是否存在，避免产生重复或不符合真实硬件的时钟定义。

UG835

五、接下来你先给我两个检查结果

先执行：

```tcl
report_clocks
report_clock_networks
```

把这两份报告的内容或截图发给我即可。

我会根据实际时钟源和约束传播情况，判断究竟是 PS 时钟配置问题、Vivado 自动约束问题，还是 RTL 时钟连接问题，再给你准确的修改方法。

我们现在的目标是先让 DIFI 模块的 `no_clock` 降为 0，并确认内部关键路径都受到有效时钟约束，然后再检查 WNS/TNS。 这比简单地让警告数量减少更重要。

你

report_clocks
INFO: [Timing 38-35] Done setting XDC timing constraints.
WARNING: [Vivado 12-25344] Design has no clocks defined matching '*'
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved. Copyright 2022-2024 Advanced Micro Devices, Inc. All Rights Reserved.

| Tool Version : Vivado v.2024.2 (win64) Build 5239630 Fri Nov 08 22:35:27 MST 2024
| Date : Mon Oct 12 15:14:35 2026
| Host : YXY_laptop running 64-bit major release (build 9200)
| Command : report_clocks
| Design : difi_tx_board_test
| Device : 7z020-clg400
| Speed File : -2 PRODUCTION 1.12 2019-11-22
| Design State : Routed

Clock Report

Attributes
P: Propagated
G: Generated
A: Auto-derived
R: Renamed
V: Virtual
I: Inverted
S: Pin phase-shifted with Latency mode

Clock Period(ns) Waveform(ns) Attributes Sources

==========
Generated Clocks
==========

==========
User Uncertainty
==========

==========
User Jitter
==========
...
显示更多

找到原因了。**你的问题不是 DIFI RTL 的时序性能不达标，而是 Vivado 当前综合、实现的顶层模块选错了。**

报告中最关键的信息是：

```
纯文本
Design       : difi_tx_board_test
Design State : Routed

WARNING: [Vivado 12-25344]
Design has no clocks defined matching '*'
```

注意 `Design : difi_tx_board_test`。

这说明你当前打开的 Implementation 结果是针对 `difi_tx_board_test` 这个模块生成的，而不是我们之前创建的完整 Zynq Block Design。

一、为什么会出现这种情况?

我们预期的工程结构应当是：

```
纯文本
difi_system_wrapper          ← FPGA 顶层
└── difi_system              ← Block Design
    ├── processing_system7_0
    ├── axi_dma_0
    ├── axi_gpio_0
    ├── axi_mem_intercon
    ├── axi_smc
    ├── proc_sys_reset_0
    └── difi_tx_board_test_0
        └── difi_data_packetizer
```

但你现在的实现结果实际上是以：

```
纯文本
difi_tx_board_test           ← 错误的 FPGA 顶层
└── difi_data_packetizer
```

为目标。

这意味着 Zynq PS、AXI DMA、AXI GPIO 等外围模块没有被纳入这次顶层实现。

`difi_tx_board_test` 单独作为顶层时，其 `aclk` 只是一个普通输入端口。由于没有对它施加有效的时钟约束，Vivado 无法知道它应该工作在 100 MHz。

因此才会出现大量 `no_clock` 和 `unconstrained_internal_endpoints` 警告。

二、如何解决?

第一步：检查真正的顶层模块

回到 Vivado 主界面，打开左侧 `Sources → Hierarchy`。

寻找：

`difi_system_wrapper`

如果已经存在：

1. 右键点击 `difi_system_wrapper`。
2. 选择 **Set as Top**。
3. 确认它变成 Sources 中的顶层模块。

如果没有 `difi_system_wrapper`，需要先右键 `difi_system.bd`，选择：

`Create HDL Wrapper → Let Vivado manage wrapper and auto-update`

生成后再设置为 Top。

第二步：通过 Tcl 确认

在 Tcl Console 输入：

```tcl
get_property top [get_filesets sources_1]
```

正确情况下应输出：

```纯文本
difi_system_wrapper
```

而不是：

```纯文本
difi_tx_board_test
```

第三步：重新运行综合与实现

这一步非常重要：之前的 Synthesis、Implementation 结果已经不能作为完整 Zynq 工程的验证结果。

修改 Top 后：

1. 右键 `Run Synthesis` 对应的 `synth_1`，选择 `Reset Runs`，确认重置综合以及依赖它的实现运行（如弹出提示）。
2. 重新执行 `Run Synthesis`。
3. 综合成功后执行 `Run Implementation`。
4. Implementation 完成后打开 `Open Implemented Design`。

如果 Vivado 已自动将旧运行标记为需要重新执行，也可以直接按提示重新运行。

第四步：重新检查时钟

再次执行：

```tcl
report_clocks
```

此时报告开头应该类似：

```纯文本
Design : difi_system_wrapper
Device : 7z020-clg400
```

时钟报告中应当能够找到 Zynq PS 提供的 `FCLK_CLK0`，其周期接近或等于 10 ns。

如果完整系统的实现仍然没有时钟，我们再检查 PS 时钟 IP 的约束生成与传播，不要预先随意添加 `create_clock`。

三、之前的 308 条 Timing 检查问题还需要单独解决吗?

暂时不需要逐条处理。

因为之前的 Implementation 根本不是针对完整 Zynq 系统进行的，所以当前的时序检查结果不能代表最终板级工程。

将顶层修改正确并重新实现后，我们再关注：

指标	目标
Design	`difi_system_wrapper`
`report_clocks`	存在 100 MHz PL 时钟
`no_clock`	0
Setup WNS	≥ 0 ns
Setup TNS	0 ns
Unconstrained Internal Endpoints	0

`no_input_delay` 和 `no_output_delay` 需要根据重新实现后的实际端口判断，不能仅凭警告数量处理。

你现在先不用管其他警告，直接在 Vivado Tcl Console 执行：

```tcl
get_property top [get_filesets sources_1]
```

如果返回的是 `difi_tx_board_test`，就按照上面的步骤将顶层改成 `difi_system_wrapper`。

这应该就是目前最主要的问题所在。

你

重新综合有严重警告：[Synth 8-6895] The reference checkpoint D:/Study/project/DIFI/vivado/difi_zynq_stage_a/difi_zynq_stage_a.srcs/utils_1/imports/synth_1/difi_tx_board_test.dcp is not suitable for use with incremental synthesis for this design. Please regenerate the checkpoint for this design with -incremental_synth switch in the same Vivado session that synth_design has been run. Synthesis will continue with the default flow

---

ChatGPT

这个警告**不代表 RTL 代码有错误，也不代表综合失败。**从日志来看，它是我们刚才修改 Vivado 顶层模块之后，旧的增量综合检查点（DCP）与新设计不匹配引起的。

而且日志最后一句非常关键：

```纯文本
Synthesis will continue with the default flow
```

意思是：**Vivado 已经放弃使用旧的增量综合结果，自动退回普通完整综合流程。**

因此，如果后续综合正常完成，这条警告本身不会影响最终网表的正确性。

一、为什么会出现?

注意报错路径：

```
纯文本
.../imports/synth_1/difi_tx_board_test.dcp
```

它引用的是旧顶层 `difi_tx_board_test` 的综合检查点。

我们刚才已经把顶层修改为：

```
纯文本
difi_system_wrapper
```

新的顶层包含整个 Zynq 系统：

- Zynq Processing System
- AXI DMA
- AXI GPIO
- AXI Interconnect
- DIFI Packetizer

而旧的 `.dcp` 只有先前的测试模块，无法作为新设计的增量综合参考。

所以这条警告符合我们刚才修改顶层之后的情况。

二、需要修复吗?

建议关闭增量综合，但如果当前综合正在运行，不需要中断。

如果最终显示 `Synthesis Completed`，而且没有其他 Error，可以继续 Implementation。这条警告不会阻止工程正常生成 Bitstream。

为了清理工程，避免后续重复出现，我建议完成当前综合后执行一次：

1. Vivado 左侧选择 `Settings`。
2. 进入 `Synthesis`。
3. 找到 `Incremental Synthesis` 选项。
4. 点击旁边的设置按钮。
5. 选择 **Disable incremental compile。**
6. 点击 OK。

这也是 AMD Vivado 综合指南 UG901 中提供的标准设置方式。

UG901 +1

如果希望彻底重新生成结果，可以随后在 Design Runs 中重置 `synth_1`，再重新运行综合。注意不要手动删除整个 `.srcs` 目录。

三、接下来重点看什么?

当前综合结束后，在 Tcl Console 执行：

```tcl
tcl
get_property top [get_filesets sources_1]
get_property STATUS [get_runs synth_1]
```

期望顶层是：

```
纯文本
difi_system_wrapper
```

如果综合成功，就继续执行 Implementation，并在打开新的 Implemented Design 后检查：

```tcl
report_clocks
report_timing_summary
```

重点确认这次不再是 `Design : difi_tx_board_test`，且确实存在 100 MHz 的 PL 时钟约束。

结论：这次的 `[Synth 8-6895]` 是旧 DCP 不适合增量综合产生的警告，无需修改任何 DIFI RTL。 现在最重要的是确认新的完整 Zynq 工程能够成功综合，并且时序约束恢复正常。

你

时序约束正常，生成比特流下一步干什么？

ChatGPT

既然你的 **Vivado Implementation 时序已经正常，而且 Bitstream 也已经成功生成**，那么 FPGA 硬件侧的第一轮构建就完成了。

接下来进入 **Vitis 2024.2 裸机开发与 Zynq 板级测试。**

我们的目标非常明确：

第一次真实板级测试

PS DDR：256 个 IQ Sample
1024 Bytes

↓ AXI DMA MM2S

PL：DIFI Data Packetizer
生成 7 Word Header + 256 Word IQ Payload

↓ AXI DMA S2MM

PS DDR：263 Word DIFI Packet
1052 Bytes · 由 ARM 逐 Word 校验

接下来的工作分为：导出 XSA、创建 Vitis Platform、创建裸机 Application、编写 DMA 测试程序、下载到 Zynq 验证。

第一步：从 Vivado 导出 XSA

在 Vivado 2024.2 顶部菜单中选择：

File → Export → Export Hardware

按照向导操作：

1. 选择 `Fixed` 硬件平台（如果出现平台类型选择）。
2. 勾选或选择 **Include Bitstream。**
3. 文件名建议填写 `difi_zynq_stage_a`。
4. 导出路径建议为 `D:\Study\project\DIFI\board\zynq7020\export`。

5. 点击 Finish。

完成后，应该得到：

`difi_zynq_stage_a.xsa`

XSA 中保存了 PS 配置、AXI IP 地址映射等硬件描述，也包含这次生成的 Bitstream。

以后只要你修改了 Vivado 硬件设计，就应该重新生成并导出 XSA，然后更新 Vitis 使用的硬件平台。

第二步：在 Vitis 2024.2 创建硬件平台

注意，你使用的是 **Vitis Unified IDE**。与旧版 Vitis Classic 不同，2024.2 通常需要先手动创建 Platform Component，再建立应用程序。
UG1400

打开 Vitis 2024.2，选择新的 Workspace，例如：

```
纯文本
D:\Study\project\DIFI\prj\vitis_ws
```

然后点击：

`File → New Component → Platform`

按照下面设置：

选项	建议设置
Component Name	`difi_zynq_platform`
Hardware Design	刚才导出的 `.xsa`
Operating System	`standalone`
Processor	`ps7_cortexa9_0`
Architecture	`32-bit`
Generate Boot Artifacts	可以勾选
Board DTSI / User DTSI	本次裸机工程先留空

你的 ZYNQ-7020 采用 Cortex-A9，因此处理器应该选择 `ps7_cortexa9_0`，而不是用于 Zynq UltraScale+ 的 `psu_cortexa53_0`。

确认后点击 Finish。

在 Vitis 的 Component Explorer 中选中刚创建的 Platform，然后通过 Flow Navigator 的 **Build**（锤子图标）构建平台。

成功后，你会获得包含 BSP、硬件驱动、处理器域及硬件描述的平台。
XD260

第三步：创建裸机 Application

平台构建成功后，点击：

`File → New Component → Application`

按照下面配置：

选项	设置
Component Name	`difi_dma_test`

Platform	`difi_zynq_platform`
Domain	`standalone_ps7_cortexa9_0`（以实际生成名称为准）
Language	C
Template	Hello World（首次测试）

具体向导可能把 Language 和 Template 放在不同页面，按实际界面选择即可。

UG1400

这里我建议**先运行一次 Hello World**，而不是立即编写 DMA 程序。

原因是，我们需要先确认 ARM 能正常运行裸机程序、JTAG 能连接目标、UART 能输出信息。

第四步：连接开发板并运行 Hello World

硬件连接方面，使用开发板原有的 JTAG 下载接口和 USB-UART 串口。确保开发板上电，串口和 JTAG 驱动正常。

在 Windows 上打开设备管理器，找到 USB-UART 对应的 COM 端口。

使用串口助手连接该端口，波特率按照 PS UART 配置；通常是 `115200`，8 数据位、无校验、1 停止位。

在 Vitis 中 Build 应用程序，成功后使用 `Run` / `Debug` 的硬件运行配置，通过 JTAG 下载运行。确认运行配置使用正确的硬件平台并完成 PS 初始化、PL 配置；有些配置会自动加载 XSA 中的 Bitstream。

串口应当能看到类似：

```
纯文本
Hello World
```

如果能够正常输出，说明 PS 裸机运行环境已经打通。

第五步：开始编写 AXI DMA 测试程序

这才是我们后续真正要开展的板级验证。

我们将把 Hello World 程序替换为 `difi_dma_test.c`，调用三个关键驱动：

驱动	作用
`xaxidma.h`	DDR 与 FPGA 之间传输 IQ 和 DIFI Packet
`xgpio.h`	控制 `start_level`，启动 Packetizer
`xil_cache.h`	处理 ARM Cache 与 DMA 访问 DDR 的一致性

尤其注意，Vitis 2024.2 的新 SDT 流程中，DMA/GPIO 初始化常使用 `xparameters.h` 里的 **BASEADDR** 宏，而不是旧教程中常见的 `DEVICE_ID`，具体以平台生成的头文件为准。

Electrical Engineering Stack Exchange +1

实际程序将先启动 S2MM 接收，再触发 Packetizer，最后启动 MM2S 发送 256 个 IQ Sample。待两路 DMA 完成后检查 Header、Timestamp 和 Payload，并通过串口输出测试结果。

现在的任务

- **Vivado Bitstream 生成**

- ◉ **导出 XSA + 构建 Vitis Platform**
- ● **Hello World 裸机测试**
- ● **DIFI DMA 板级测试**

你现在先完成 XSA 导出、Vitis Platform 创建和 Hello World 测试。

等这三个步骤完成，我们就正式编写适配当前硬件的 `difi_dma_test.c`，通过 AXI DMA 把第一包 DIFI 数据真正跑通。这一步成功后，才算完成我们竞赛项目的第一次 FPGA 板级功能验证。

你

你直接写一个可以用的hello world .c程序供我测试

ChatGPT

已经写好了一份适用于 **ZYNQ-7020 + Vitis 2024.2 + Standalone 裸机环境**的 Hello World 程序。

difi_hello_world.c

C 语言 · ARM Cortex-A9 · UART 串口测试

下载完整 C 文件 ↗

一、完整代码

C

```c
#include "xil_printf.h"
#include "sleep.h"

int main(void)
{
    unsigned int counter = 0U;

    xil_printf("\r\n");
    xil_printf("========================================\r\n");
    xil_printf(" DIFI Zynq-7020 Board Test\r\n");
    xil_printf(" PS UART Hello World\r\n");
    xil_printf("========================================\r\n");
    xil_printf("Hello World!\r\n");
    xil_printf("PS bare-metal application started.\r\n");
    xil_printf("If you can read this, UART output works.\r\n\r\n");

    while (1) {
        xil_printf("[DIFI] PS running, counter = %u\r\n", counter++);
        sleep(1);
    }

    return 0;
}
```

这里使用了两个 Xilinx 提供的函数：

- `xil_printf()`：通过 BSP 配置的标准输出设备发送串口信息。
- `sleep(1)`：延迟约 1 秒，让程序每秒输出一条运行信息。

二、在 Vitis 中使用

1. 打开你刚才创建的 `difi_dma_test` Application Component。

2. 进入 `Sources`，找到原来的 `hello_world.c` 或 `main.c`。
3. 用上述代码**替换原文件内容**，不要保留两个 `main()`。
4. 点击 **Build**，确认编译成功。
5. 连接开发板的 JTAG 和 USB-UART，打开串口助手。
6. 串口波特率设置为 `115200`（前提是 BSP 的 stdout UART 使用这个波特率），数据格式为 `8N1`。
7. 通过 Vitis 的硬件运行配置下载并启动程序。

三、预期输出

成功后，串口应出现：

```
纯文本
========================================
 DIFI Zynq-7020 Board Test
 PS UART Hello World
========================================
Hello World!
PS bare-metal application started.
If you can read this, UART output works.

[DIFI] PS running, counter = 0
[DIFI] PS running, counter = 1
[DIFI] PS running, counter = 2
[DIFI] PS running, counter = 3
...
```

如果串口每秒持续输出递增数字，说明 ARM 裸机程序已经正常运行，BSP 的 UART 标准输出和延时功能也基本正常。

注意：这个程序只验证 PS 和 UART，还没有测试 PL、AXI DMA 或 DIFI 协议逻辑。

等这一步通过，我们就开始写正式的 `difi_dma_test.c`，让 PS 通过 DMA 发送 256 个 IQ Sample，接收 FPGA 生成的完整 DIFI Packet，并自动进行逐 Word 比对。

difi_hello_world.c
代码

你

这一步没问题

ChatGPT

Hello World 已经通过，说明 **Zynq PS 裸机程序运行、JTAG 下载和 UART 串口输出**都正常。

现在进入我们竞赛项目的第一个关键里程碑：**让 FPGA 真正生成一包 DIFI 数据，并由 PS 读回验证。**

本次测试的数据通路是：

`DDR（1024 B IQ）→ DMA MM2S → DIFI Packetizer → DMA S2MM → DDR（1052 B Packet）`

我已经核对了之前给你的 `difi_tx_board_test.v` 和 GitHub 上的 Packetizer 实现。它们要求通过 GPIO 的 `0→1` 上升沿启动一次封包，且第一轮固定为每包 256 个 IQ 样本。

测试程序必须遵循这个顺序：**先启动 S2MM 接收，再触发 GPIO，最后启动 MM2S 发送。** 这样才能让 DMA 在 Packetizer 输出数据前准备好接收缓冲区。

一、下载板级 DMA 测试程序

difi_dma_test.c
Vitis 2024.2 · Standalone · AXI DMA Simple Mode

下载完整 C 源程序↗

程序采用 AMD 官方 AXI DMA 轮询驱动和 AXI GPIO 驱动接口，兼容 Vitis 2024.2 常用的 SDT 平台配置方式。
GitHub +1

它会自动完成 DMA/GPIO 初始化、测试数据生成、Cache 维护、DMA 双向传输、超时检测及逐 Word 校验。

我已对代码做过基础语法检查，但还没有在你的实际硬件平台上运行，因此板级结果仍需你验证。

二、如何使用?

1. 打开 Vitis 中的 `difi_dma_test` Application Component。
2. 将原来的 `hello_world.c` 替换为下载的 `difi_dma_test.c`，确保整个应用中只有一个 `main()`。
3. 确认 BSP 已包含 `axidma`、`gpio` 驱动；如果提示找不到 `xaxidma.h` 或 `xgpio.h`，需要检查平台 BSP。
4. 点击 Build，检查是否成功。
5. 保持开发板 JTAG 和串口连接，确保运行配置会加载刚才生成的 DIFI Bitstream。
6. Run Application，观察串口结果。

三、正确的运行结果

成功时，串口应该出现类似以下内容：

纯文本

```
=======================================
 DIFI Stage-A / Zynq-7020 DMA Test
 IQ: 1024 bytes -> DIFI: 1052 bytes
=======================================
[INIT] TX buffer = 0x........
[INIT] RX buffer = 0x........
[START] S2MM armed (1052 B)
[START] GPIO start pulse sent
[START] MM2S armed (1024 B)
[OK] MM2S complete
[OK] S2MM complete

[RX] First 9 words:
     word[0] = 0x18600107
     word[1] = 0x00000001
     word[2] = 0x006a621e
     word[3] = 0x00000000
     word[4] = 0x6553f100
     word[5] = 0x00000000
     word[6] = 0x00000000
     word[7] = 0x1000a000
     word[8] = 0x1001a003

[RX] Packet sequence = 0
[PASS] All 7 header words and 256 IQ payload words match!

[DIFI STAGE-A TEST PASSED]
```

这里的 Sequence = 0 是 FPGA 刚复位后的预期值；如果 PL 没有重新复位，后续发包的 Sequence 可以继续递增。

四、如果运行失败怎么办?

程序会输出具体错误位置。

现象	优先排查方向
编译时找不到 DMA/GPIO 地址宏	Vitis BSP、`xparameters.h`
DMA 初始化失败	XSA 与当前 Bitstream 是否对应
MM2S Timeout	GPIO 启动信号、PL 时钟、AXIS 背压

S2MM Timeout	`TLAST`、AXIS 数据连接、S2MM 接收路径
DIFI Header Mismatch	Packetizer 输出、寄存器数据与预期配置
IQ Payload Mismatch	样本顺序、DMA 缓冲区、Cache 一致性

如果出现 Timeout，程序还会打印 MM2S 和 S2MM 的 `DMASR` 状态寄存器，有助于进一步定位。

xilinx.github.io

你现在直接编译、运行这份程序就可以。 如果输出 `[DIFI STAGE-A TEST PASSED]`，意味着我们已完成第一轮真正的 PS → DMA → PL Packetizer → DMA → PS 板级数据验证。

到时我们就可以将单独的 Data Packetizer 替换成完整 `difi_engine_top`，开始三类 DIFI Packet 的板级收发和配置测试。

difi_dma_test.c
代码

你

```
------------------------------------------------------------------------------
[2026/10/12 下午3:55:05]: Clean for difi_dma_test::clean with id '3160c914-85ce-470d-b02c-897294f4418d' started.
------------------------------------------------------------------------------
-- Configuring done
CMake Error at CMakeLists.txt:60 (add_executable):
  Cannot find source file:

    difi_hello_world.c

  Tried extensions .c .C .c++ .cc .cpp .cxx .cu .mpp .m .M .mm .ixx .cppm .h
  .hh .h++ .hm .hpp .hxx .in .txx .f .F .for .f77 .f90 .f95 .f03 .hip .ispc


CMake Error at CMakeLists.txt:60 (add_executable):
  No SOURCES given to target: difi_dma_test.elf


CMake Generate step failed.  Build files cannot be regenerated correctly.
[0/1] D:\software\xilinx_2024.2\Vitis\2024.2\tps\win64\cmake-3.24.2\bin\cmake.exe --regenerate-during-build -
SD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\src -BD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\build
FAILED: build.ninja
D:\software\xilinx_2024.2\Vitis\2024.2\tps\win64\cmake-3.24.2\bin\cmake.exe --regenerate-during-build -
SD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\src -BD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\build
[ERROR] ninja: error: rebuilding 'build.ninja': subcommand failed
[ERROR] difi_dma_test build failed.
------------------------------------------------------------------------------
------------------------------------------------------------------------------
[2026/10/12 下午3:55:52]: Clean for difi_dma_test::clean with id '8d0d769a-e5b4-4a06-979e-602d3ad68e6d' started.
------------------------------------------------------------------------------
-- Configuring done
CMake Error at CMakeLists.txt:65 (add_executable):
  Cannot find source file:

    difi_hello_world.c

  Tried extensions .c .C .c++ .cc .cpp .cxx .cu .mpp .m .M .mm .ixx .cppm .h
  .hh .h++ .hm .hpp .hxx .in .txx .f .F .for .f77 .f90 .f95 .f03 .hip .ispc
```

CMake Error at CMakeLists.txt:65 (add_executable):
  No SOURCES given to target: difi_dma_test.elf

CMake Generate step failed. Build files cannot be regenerated correctly.
[0/1] D:\software\xilinx_2024.2\Vitis\2024.2\tps\win64\cmake-3.24.2\bin\cmake.exe --regenerate-during-build -SD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\src -BD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\build
FAILED: build.ninja
D:\software\xilinx_2024.2\Vitis\2024.2\tps\win64\cmake-3.24.2\bin\cmake.exe --regenerate-during-build -SD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\src -BD:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\build
[ERROR] ninja: error: rebuilding 'build.ninja': subcommand failed
[ERROR] difi_dma_test build failed.
----------编译报错了，怎么回事
...

显示更多

ChatGPT

这个报错原因很明确：**你替换了 C 源文件，但 Vitis 2024.2 的 CMake 构建配置仍然引用旧文件 `difi_hello_world.c`，导致找不到源文件。**

这不是 AXI DMA、Zynq 硬件平台或 C 代码本身的问题，甚至还没有进行到 C 语言编译阶段。

一、为什么会报错?

看日志中最关键的两行：

```
纯文本
Cannot find source file:
    difi_hello_world.c

No SOURCES given to target: difi_dma_test.elf
```

说明你的 `CMakeLists.txt` 中仍然保留着之前 Hello World 测试程序的文件名。

很可能你现在的目录是：

```
纯文本
difi_dma_test/
└── src/
    ├── CMakeLists.txt
    └── difi_dma_test.c          ← 现在的新文件
```

但 CMake 仍在寻找：

```
纯文本
difi_hello_world.c               ← 原来的文件
```

因此无法构建。

二、怎么解决?

我建议采用最简单的方法，不需要修改 CMake 配置。

方法一：直接恢复原来的文件名（推荐）

1. 打开 Vitis 的 `difi_dma_test` Application。
2. 找到 `src/difi_dma_test.c`。
3. 右键选择 `Rename`，把文件名修改为：

   `difi_hello_world.c`
4. 保留新 DMA 测试程序的全部代码，不需要把代码改回 Hello World。
5. 保存，然后重新执行 Build。

C 文件名不必与 Application 名称一致。 即使文件叫 `difi_hello_world.c`，内部完全可以是 AXI DMA 测试程序。

这样不需要调整 CMakeLists.txt，因为其引用的文件名重新存在了。

方法二：修改 CMakeLists.txt（如果想统一命名）

如果希望正式使用 `difi_dma_test.c` 这个名称，可以打开：

```
纯文本
D:\Study\project\DIFI\vitis\vitis_ws\difi_dma_test\src\CMakeLists.txt
```

找到包含 `difi_hello_world.c` 的位置，例如：

```
cmake
add_executable(difi_dma_test.elf
    difi_hello_world.c
)
```

将源文件名改为：

```
cmake
add_executable(difi_dma_test.elf
    difi_dma_test.c
)
```

实际 `add_executable` 可能还包含其他参数或源文件，**只修改源文件名即可，不要替换整个命令。**

保存后重新 Build。

不过 Vitis Unified IDE 的部分 CMake 配置由工具管理，后续可能重新生成。如果发现修改被覆盖，应通过 Vitis 的应用源文件管理功能更新源文件列表，或者直接采用方法一。

三、为什么 Clean 也会报错?

你日志中显示的是：

```
纯文本
Clean for difi_dma_test::clean
```

因为 Vitis 底层使用 CMake + Ninja，执行 Clean 时也可能触发 CMake 配置重新生成。

旧文件找不到，所以连 Clean 都失败了。

这并不意味着工程损坏。

现在你只需要把 `difi_dma_test.c` 重命名为 `difi_hello_world.c`，然后重新 Build。

如果之后出现新的编译错误，例如 `XPAR_XAXIDMA_0_BASEADDR` 未定义或 `XAxiDma_CfgInitialize` 参数不匹配，把新的报错日志发过来，我们再针对 Vitis 2024.2 的 BSP 和 DMA 驱动接口进行修改。

你

没问题，测试通过：
```
========================================
 DIFI Stage-A / Zynq-7020 DMA Test
 IQ: 1024 bytes -> DIFI: 1052 bytes
========================================
[INIT] TX buffer = 0x0010C140
[INIT] RX buffer = 0x0010C540
[START] S2MM armed (1052 B)
[START] GPIO start pulse sent
[START] MM2S armed (1024 B)
[OK] MM2S complete
[OK] S2MM complete
[DMA] MM2S_DMASR=0x00001002  S2MM_DMASR=0x00001002
[RX] First 9 words:
    word[0] = 0x18600107
    word[1] = 0x00000001
    word[2] = 0x006A621E
    word[3] = 0x00000000
    word[4] = 0x6553F100
    word[5] = 0x00000000
    word[6] = 0x00000000
    word[7] = 0x1000A000
    word[8] = 0x1001A003
[RX] Packet sequence = 0
[PASS] All 7 header words and 256 IQ payload words match!

[DIFI STAGE-A TEST PASSED]
...
```
显示更多

ChatGPT

Stage A 板级测试正式通过！ 这是我们 DIFI 竞赛项目的一个重要里程碑：第一次在真实 Zynq-7020 硬件上完成了 DIFI Signal Data Packet 的生成与验证。

一、这次测试证明了什么?

DIFI Stage A — Board Test ✓ PASS

IQ 输入	DIFI 输出
1024 B	**1052 B**
Payload 样本	数据比较
256/256	**全部一致**

✓ AXI DMA MM2S / S2MM 均完成传输

- DIFI Header、时间戳、Stream ID 正确
- 256 个 IQ 样本顺序与数值完全一致

尤其值得注意的是：

```
纯文本
MM2S_DMASR = 0x00001002
S2MM_DMASR = 0x00001002
```

这表示两路 DMA 的 Idle 位与 IOC（传输完成中断状态）位置位，没有报告 DMA 错误状态。

以及：

```
纯文本
word[0] = 0x18600107
```

其中 Packet Size = `0x0107` = 263 Words，正好等于 7 Words 协议头 + 256 Words IQ Payload。

现在我们已经能够确认：第一版 Data Packetizer 不仅通过了 RTL 仿真，而且在真实 FPGA 上也能够按照预期工作。

不过，这还不能代表完整 DIFI Engine、RX Parser 或 UDP 网络通信已经通过板级验证。

二、下一步：Stage B，完整 DIFI Engine 上板

我建议不要继续停留在单独的 Data Packetizer，而是开始接入之前完成的 `difi_engine_top`。

采用渐进式集成，避免一次引入所有功能。

阶段	测试内容	验收标准
Stage A	单独 Data Packetizer 上板	已通过
Stage B1	完整 Engine 的 TX、AXI4-Lite 配置	PS 收到 Version、Context、Data 三类 Packet
Stage B2	完整 Engine 的 RX Parser	PS 发送 DIFI Packet，接收解析后的 IQ、Metadata
Stage B3	TX → RX 板内闭环	IQ 经封包、解析后逐样本一致
Stage C	PC UDP / DIFI 外部验证	真实网络互通、Wireshark 与外部工具验证

Stage B1 需要做的改动

相比刚才的 Block Design，下一阶段主要变化有三点：

1. 将 `difi_tx_board_test` 替换为一个新的板级 Wrapper，内部实例化完整的 `difi_engine_top`。
2. 将 PS 的 AXI GP0 经互联连接到 `difi_engine_top` 已有的 **AXI4-Lite Slave**，使 PS 能够直接设置 Stream ID、采样率、Packet Size 和 TX Enable。
3. 增加简单的板级时间源、周期触发和 `data_available` 适配逻辑，同时处理暂时不用的 RX 接口和 Metadata 握手。

还要特别注意：完整引擎启动后会优先输出 **Version → Context → Data**。这三类包分别具有自己的 `TLAST`，不能像 Stage A 那样直接让一次 Simple Mode DMA 接收把全部三类包当作一个包。第一版可以通过逐包启动 S2MM 来验证。

三、当前最值得保存的成果

建议将本次串口日志保存为：

`board/zynq7020/tests/stage_a_dma_pass.log`

同时保存对应的 Bitstream、XSA、Vitis 测试源码和 Vivado 时序报告。

以后更新 RTL 或改动 Block Design 时，可以用 Stage A 作为硬件回归测试基线，判断是否引入了新的问题。

我的建议是下一步直接开始 Stage B1：基于当前已经验证成功的 Vivado 工程，建立完整 `difi_engine_top` 的板级 Wrapper，并连接 AXI4-Lite + DMA。

这样我们就真正从单一模块板级测试，迈向整个 DIFI 1.1 协议引擎的系统级硬件验证。

你

如何保存串口日志?

ChatGPT

如果你使用的是之前提到的 **MobaXterm** 查看 Zynq 串口输出，保存日志非常简单，有两种方法。

方法一：直接保存当前串口内容（推荐）

适合保存刚才已经通过的 Stage A 测试结果。

1. 在 MobaXterm 中切换到显示 DIFI 测试输出的串口终端。
2. 点击顶部菜单 **Terminal → Save terminal text。**
3. 选择保存目录。
4. 文件类型选择纯文本（Plain Text）。
5. 保存为 `stage_a_dma_pass.log` 。

MobaXterm 官方文档也推荐通过这个功能直接导出终端文本。

Documentation

建议保存到：

```
纯文本
D:\Study\project\DIFI\
└── board\
    └── zynq7020\
        └── tests\
            └── stage_a_dma_pass.log
```

`.log` 本质上就是文本文件，可以直接使用 VS Code 打开。保存后检查一下，确认包含完整的 `[DIFI STAGE-A TEST PASSED]` 输出。

方法二：自动记录后续测试日志

后续我们还要进行多轮 DMA、Context、Version 和性能测试，建议开启自动日志记录。

在 MobaXterm 中：

1. 点击 **Settings → Configuration → Terminal。**
2. 勾选 `Log terminal output to the following directory` 。
3. 设置日志目录，例如 `D:\Study\project\DIFI\board\zynq7020\tests` 。
4. 如有 Log Type 选项，建议选择 `Printable output with timestamps` 。
5. 保存设置，重新打开串口会话，使设置生效。

这样后续串口输出就可以自动写入日志。

mobaxterm.com.cn

我的建议： 当前 Stage A 的 PASS 日志使用方法一直接保存。后续做 Stage B 和长时间稳定性测试时，再开启自动日志记录，并将每次测试的日志与对应 Bitstream/XSA 版本关联保存。

如果你实际上使用的是 Vitis 内置 Serial Terminal，而不是 MobaXterm，告诉我，我再按 Vitis 2024.2 的界面教你操作。

你

开始Stage B1

ChatGPT

我们正式开始 **Stage B1：完整 DIFI Engine 的 TX 通路与 AXI4-Lite 配置上板测试。**

这次不再测试单独的 Data Packetizer，而是将之前完成的完整 `difi_engine_top` 接入 Zynq 系统，验证三类 DIFI Packet 的生成、调度以及 PS 寄存器配置。

我已经对照 GitHub 当前 `feature/rtl-integration` 分支的 `difi_engine_top.v`，为你编写了新的板级 Wrapper，并核对其与 Top 的全部 80 个端口对应关系。

一、下载 Stage B1 的 RTL

difi_engine_board_b1.v

Verilog-2001 · 完整引擎 TX 板级适配

下载 RTL 文件 ↗

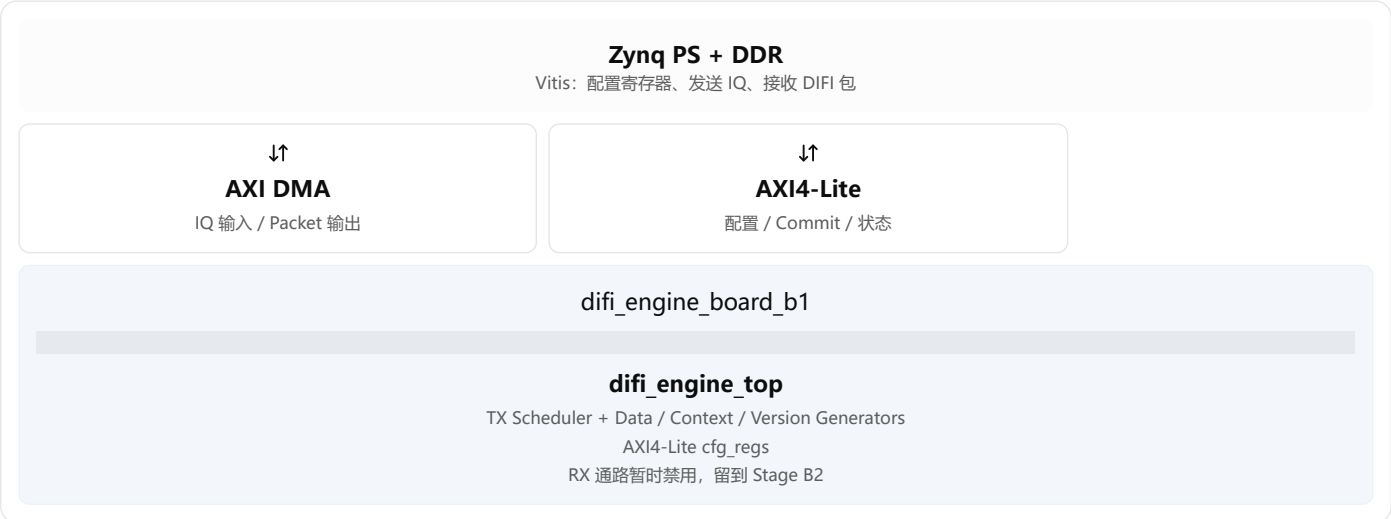
Stage B1 工程包

Wrapper + 集成说明 + 寄存器与测试要求

下载 ZIP ↗

这份 Wrapper 已完成接口对照，但尚未在你的 Vivado 工程中综合或进行真实板级验证。

二、Stage B1 的目标结构



这里有两个与 Stage A 的区别：

第一，不再需要 GPIO 触发封包。 完整引擎通过 TX Scheduler 自动调度发送。Wrapper 使用输入 IQ Stream 的 `TVALID` 指示数据可用。

第二，完整引擎首次使能后会按顺序发送 Version、Context、Data。 Stage B1 先关闭周期性触发，只测试首次自动发送，避免引入不必要的复杂性。

三、现在修改 Vivado Block Design

先备份 Stage A 已经成功运行的 Vivado 工程，避免覆盖测试基线。

第一步：添加新的 RTL

在本地仓库中创建：

```
纯文本
DIFI/
└── rtl/
    └── board/
        └── difi_engine_board_b1.v
```

将下载的 `.v` 文件放入其中。

在 Vivado 中通过 `Add Sources` 添加新文件，同时确保 `difi_engine_top.v` 及其全部九个下层 RTL 模块已经加入工程，并包含两个头文件：

- `difi_v1_1_defs.vh`
- `difi_cfg_regmap.vh`

第二步：替换旧 Wrapper

打开原来的 `difi_system.bd`。

删除原来的 `difi_tx_board_test_0` 模块实例，保留 AXI DMA、Zynq PS、互联模块、时钟和复位 IP。

然后右键 Block Design 空白处，选择：

Add Module → difi_engine_board_b1

成功后应当出现三个主要接口：

`S_AXIS_IQ`、`M_AXIS_PKT`、`S_AXI_CTRL`。

第三步：连接接口

来源	连接目标
`axi_dma_0/M_AXIS_MM2S`	`difi_engine_board_b1_0/S_AXIS_IQ`
`difi_engine_board_b1_0/M_AXIS_PKT`	`axi_dma_0/S_AXIS_S2MM`
PS `M_AXI_GP0` 经互联	`difi_engine_board_b1_0/S_AXI_CTRL`
`FCLK_CLK0`	`difi_engine_board_b1_0/aclk`
`peripheral_aresetn`	`difi_engine_board_b1_0/aresetn`

新的 `S_AXI_CTRL` 是 **AXI4-Lite Slave**，需要由 PS 的 GP0 主口访问。可用 `Run Connection Automation` 自动创建或扩展 AXI Interconnect/SmartConnect。

Stage A 中的 AXI GPIO 不再需要参与启动。可以暂时保留 IP，但不要将它的输出接入新的 Wrapper。

第四步：检查 Address Editor

进入 Vivado 的 `Address Editor`，确认 PS 能够访问新加入的 DIFI AXI4-Lite Slave，并分配了与 DMA/GPIO 不重叠的地址空间。

我们的寄存器偏移为：

偏移地址	寄存器	功能
`0x00`	CONTROL	TX Enable
`0x04`	COMMAND	Config Commit
`0x08`	STATUS	TX Busy、Commit Pending 等

`0x10`	DATA_STREAM_ID	数据流 SID
`0x14`	VERSION_STREAM_ID	Version SID
`0x18`	SAMPLES_PER_PACKET	每包 IQ 数量
`0x1C`	SAMPLE_RATE_HZ	采样率
`0x20`	BANDWIDTH_HZ	带宽
`0x40` 起	ACTIVE 配置读回	核对 Commit 是否生效

这些是相对基地址的偏移，不是 ARM 端的绝对地址。

请记下 Vivado 分配给 `S_AXI_CTRL` 的 Base Address。 后续 Vitis 程序要用它访问配置寄存器。

四、Stage B1 最终要验证什么?

由于 AXI DMA Simple Mode 一次接收通常对应一个由 `TLAST` 结束的数据包，我们不会把三类 Packet 当作一次 DMA 接收。

发送顺序	Packet	字节数	第一个 Word（Seq=0）
1	Version Context	44 B	`0x5960000B`
2	Signal Context	108 B	`0x4960001B`
3	Signal Data	1052 B	`0x18600107`

Vitis 程序会分别为三类 Packet 准备接收事务，并验证字段正确性。Scheduler 遇到下游 DMA 尚未准备好时，应通过 AXI4-Stream 背压等待。

另外，当前 Wrapper 仍使用固定测试时间戳，RX 输入也暂时被禁用。因此 Stage B1 只证明完整引擎的 **TX、调度和配置功能**，RX 解析留到 Stage B2 验证。

五、这一步先做到哪里?

我们仍按之前的方式，一次解决一个环节。

你现在先完成：添加新 RTL → 替换旧模块 → 连接 AXI4-Stream 和 AXI4-Lite → Validate Design。

完成后给我发两张截图：一张完整的 Block Design，另一张 Address Editor 中新 DIFI Slave 的基地址。

我会先检查连接和地址映射，然后带你重新生成 Bitstream/XSA，编写 Stage B1 对应的 Vitis 测试程序，实现 **Version → Context → Data 三类包的真实 FPGA 硬件验证。**

你




ChatGPT

我检查了你发来的 **Block Design 和 Address Editor**。结论是：**Stage B1 的硬件连接基本正确，地址映射也没有发现冲突，可以进入下一步。**

一、Block Design 检查结果

检查项目	结果
DMA `M_AXIS_MM2S` → DIFI `S_AXIS_IQ`	✓ 正确
DIFI `M_AXIS_PKT` → DMA `S_AXIS_S2MM`	✓ 正确
PS GP0 → DIFI `S_AXI_CTRL`	✓ 正确
DMA MM2S / S2MM → PS HP0	✓ 正确

100 MHz 时钟与低有效复位	✅ 连线正确
AXI GPIO	✅ 保留但不再参与启动
`tx_busy` 未连接	✅ 允许暂时悬空

这次相比 Stage A，多出来的 AXI4-Lite 配置通路也已经正确接入 SmartConnect。

二、Address Editor 检查结果

你目前的基地址如下：

外设	基地址	地
DIFI Engine	`0x40000000`	4
AXI DMA	`0x40400000`	6
AXI GPIO	`0x41200000`	6
DDR（经 HP0）	`0x00000000` 起	1

这些地址没有冲突。

接下来 Vitis 程序访问 DIFI 配置寄存器时，以 `0x40000000` 为基地址。例如：

```c
#define DIFI_BASE_ADDR 0x40000000U

// CONTROL: TX Enable
Xil_Out32(DIFI_BASE_ADDR + 0x00, 1);

// STATUS: Read current status
u32 status = Xil_In32(DIFI_BASE_ADDR + 0x08);

// DATA_STREAM_ID: shadow config
Xil_Out32(DIFI_BASE_ADDR + 0x10, 0x12345678);
```

注意，最后一项只是写入 Shadow 寄存器，**还必须通过 `COMMAND` 的 Commit 操作，才能让新配置变成 Active 配置。**

当前分配的 4 KB DIFI 地址窗口比内部实际使用的 256 字节寄存器空间大，但第一次板级测试可以正常使用。后续如有需要再收紧地址映射。

三、接下来开始生成 Stage B1 的 Bitstream

这一步和 Stage A 基本相同，但有三个需要特别注意的地方。

1. **确认 FPGA 顶层仍为 `difi_system_wrapper`**，不要把 `difi_engine_board_b1` 设置成整个工程的顶层。可以在 Tcl Console 中执行：

   ```tcl
   get_property top [get_filesets sources_1]
   ```

2. **确认完整 Engine 所需的 RTL 都已添加。** 包括六个协议生成/解析模块、TX Scheduler、RX Dispatch、cfg_regs、engine_top，以及 `difi_engine_board_b1.v`。两个 `.vh` 头文件也要能够被找到。
3. 依次执行：

   `Generate Output Products → Run Synthesis → Run Implementation → Generate Bitstream`

   综合完成后留意是否存在未解析模块（Black Box）或大量意外被优化掉的逻辑；Implementation 完成后确认 100 MHz 时钟约束存在、WNS ≥ 0、TNS = 0。

Bitstream 生成成功后，再次执行：

File → Export → Export Hardware → Include Bitstream

将新 XSA 命名为 `difi_zynq_stage_b1.xsa`，不要覆盖 Stage A 的验证基线。

四、Stage B1 的 Vitis 测试顺序

这次 Vitis 程序与 Stage A 不同，不能继续使用原来的 GPIO 启动程序。

我们准备按下面的顺序测试：

1. 通过 AXI4-Lite 读写 DIFI 寄存器，检查 Shadow → Active Commit 是否成功。
2. 启动 S2MM 接收，设置 TX Enable，接收第一个 **Version Packet（44 Bytes）**。
3. 重新启动 S2MM，接收第二个 **Context Packet（108 Bytes）**。
4. 重新启动 S2MM 接收 1052 Bytes，再启动 MM2S 发送 256 个 IQ 样本，接收 **Data Packet（1052 Bytes）**。
5. 逐 Word 检查三类 Packet 的 Header、Stream ID、时间戳、Context 配置字段，以及 256 个 IQ 样本。

这里最关键的是 **DMA S2MM 必须按三个独立 Packet 分别接收。**不能简单设置 1204 Bytes，让 DMA 一次接收全部三类包，因为每个 Packet 都有自己的 `TLAST`。

另外，当前 Stage B1 Wrapper 使用固定测试时间戳，不是实时 UTC 时间；这个阶段主要验证协议生成、调度和可配置性。

现在可以继续生成 Bitstream 和新的 XSA。 完成后，我们就编写专门的 `difi_stage_b1_test.c`，沿用已经验证成功的 DMA 驱动结构，增加 AXI4-Lite 配置和三包自动校验。

你

新的xsa成功生成

ChatGPT

新的 XSA 已成功生成，接下来进入 **Stage B1 的 Vitis 板级测试。**

这次我们要验证的不再是单独的 Data Packetizer，而是完整 DIFI Engine 的 TX Scheduler、三类 Packet Generator 和 AXI4-Lite 配置功能。

一、先更新 Vitis 硬件平台

不需要重新创建整个 Vitis Workspace，可以直接更新原来的 Platform Component。

在 Vitis 2024.2 中：

1. 找到之前创建的 `difi_zynq_platform`。
2. 右键该 Platform，展开 `Settings`，打开 `vitis-comp.json`。
3. 点击 **Switch XSA。**
4. 选择刚才生成的 `difi_zynq_stage_b1.xsa`。
5. 重新 Build Platform，再重新 Build Application。

这是 AMD 官方 UG1400 对 Vitis 2024.2 的更新流程。

UG1400

二、下载 Stage B1 测试程序

我已经根据当前 GitHub 仓库中的 `difi_engine_top`、`difi_cfg_regs`、TX Scheduler、Context 和 Version Generator 的实际逻辑，编写了测试程序。

difi_stage_b1_test.c

〈/〉 Vitis 2024.2 · Zynq-7020 · Standalone · 约 14 KB

下载完整 C 测试程序↗

程序完成以下验证：

测试项目	检查内容
AXI4-Lite	Shadow 寄存器写入、Commit、Active 读回
Version Packet	11 Words、Header、版本信息
Context Packet	27 Words、Fs、BW、RF、DPF 等
Data Packet	7 Words Header + 256 IQ Sample
AXI DMA	分三次 S2MM 接收、一次 MM2S 发送
数据正确性	逐 Word 与预期结果比较

其中 DIFI 配置接口的基地址，已经根据你刚才的 Address Editor 设置为 `0x40000000`。

将程序放入 Vitis

你之前遇到过 CMake 找不到旧 C 文件名的问题，所以这次最稳妥的办法是：

打开现有的 `difi_hello_world.c`，直接把全部内容替换为新程序的代码，不修改文件名。

然后保存并 Build。

三、运行之前注意一件事

请确保 Vitis 硬件运行配置加载的是 Stage B1 的新 Bitstream，并重新配置 PL。

原因是 TX Scheduler 中有一个首次发送标志，Reset 后才会按照：

`Version → Context → Data`

的顺序进行首次发包。

只重新启动 ARM 程序，而不重新复位 PL，可能导致 Version Packet 不再自动发送，使第一次 DMA 接收超时。

四、预期串口结果

成功时，应该看到类似：

```
纯文本
DIFI Stage-B1: TX + AXI4-Lite + AXI DMA
Version(44B) -> Context(108B) -> Data(1052B)

[CFG] Initial STATUS=0x00000012
[OK] Active Data SID=0x10203040
[OK] Active Version SID=0x55667788
...
[PASS] AXI4-Lite Shadow->Active commit

[START] Version S2MM armed (44 B)
[START] Engine TX enabled
[OK] Version received
[PASS] Version Packet: 11 words

[START] Context S2MM armed (108 B)
[OK] Context received
[PASS] Context Packet: 27 words, BW/RF/Fs/DPF match

[START] Data S2MM armed (1052 B)
[START] MM2S armed (1024 B IQ)
[OK] Data received
[PASS] Data Packet: 7 header + 256 IQ samples
```

```
[DIFI STAGE-B1 TEST PASSED]
```

实际输出中还会显示各包的 Header、Stream ID 和 DMA 状态，部分状态寄存器数值可能随运行时机变化。

这里特意使用了非默认 Stream ID：

- Data SID：`0x10203040`
- Version SID：`0x55667788`

这样能够验证配置数据确实经过 AXI4-Lite 写入、Commit，最终进入 FPGA 生成的 Packet，而不是恰好使用了复位默认值。

你现在先更新 Vitis Platform，替换 C 代码，Build 并运行。

这份程序已完成基础 C 语法检查，但尚未在你的 Zynq 实机上验证。如果出现编译错误、DMA Timeout 或 Packet Mismatch，把原始日志发给我，我们直接定位对应模块。成功之后，就可以将 Stage B1 记录为完整协议引擎 TX 方向的首次板级验证通过。

‹/› difi_stage_b1_test.c
代码

你

如果我不想直接替换掉旧的difi_stage_a_test.c，我该怎么做？再新建一个application吗？还是再该application的src目录下创建difi_stage_b1_test.c?

ChatGPT

我更推荐你**为 Stage B1 新建一个独立的 Application Component**，而不是把 `difi_stage_b1_test.c` 直接放到原来 Application 的 `src` 目录里。

原因很简单：我们后续还要开展 Stage B2、Stage B3 等测试，每个阶段都应该保留自己的源码、编译结果和测试记录，方便回归测试和竞赛复现。

一、推荐的 Vitis 工程组织方式

Vitis Workspace

difi_zynq_platform
当前 Stage B1 的 XSA 硬件平台

difi_stage_a_test
src/difi_stage_a_test.c — 已通过板级测试，保留

difi_stage_b1_test
src/difi_stage_b1_test.c — 即将创建

difi_stage_b2_test
未来 RX Parser 板级测试

每个 Application 都有自己的 `src`、CMake 配置和 ELF 文件，互不干扰。

二、如何新建 Stage B1 Application?

1. 在 Vitis 2024.2 中选择 `File → New Component → Application`。
2. Component Name 填写 `difi_stage_b1_test`。
3. Platform 选择已经更新 XSA 的 `difi_zynq_platform`。
4. Domain 选择 `standalone_ps7_cortexa9_0`（以实际名称为准）。

5. Template 优先选择 `Empty Application`；如果没有，就使用 `Hello World` 模板。
6. 点击 Finish，完成创建。

随后进入新 Application 的 `src` 目录，加入下载好的 `difi_stage_b1_test.c`。

特别注意 CMakeLists.txt：

如果生成的 `CMakeLists.txt` 仍然引用 `main.c` 或 `hello_world.c`，需要通过 Vitis 的源文件管理界面更新文件列表，或将对应源文件名改成：

```cmake
difi_stage_b1_test.c
```

同时确保这个 Application 中只有一个参与编译的 `main()` 函数，避免重复定义。你之前遇到的 CMake 错误就是文件名与构建配置不一致导致的。

最后 Build 新的 Application 即可。

三、一个容易忽略的问题：硬件平台版本

虽然两个 Application 可以共用一个 Platform，但**Stage A 和 Stage B1 使用的 FPGA 硬件设计实际上不同。**

项目	Stage A	Stage B1
FPGA 逻辑	单独 Data Packetizer	完整 DIFI Engine
控制方式	AXI GPIO	AXI4-Lite
XSA	`difi_zynq_stage_a.xsa`	`difi_zynq_stage_b1.xsa`
Application	`difi_stage_a_test`	`difi_stage_b1_test`

因此，如果你希望以后能够方便地重新运行 Stage A，**更规范的方案是连 Platform 也分别保存**：

- `difi_platform_stage_a` → Stage A XSA
- `difi_platform_stage_b1` → Stage B1 XSA

两个 Application 分别依赖对应的 Platform。这样以后重新生成某个阶段的硬件平台，不会影响另一个阶段的构建环境。

已经通过测试的 Stage A 工程可以先保留原样，不需要现在重建。

我的建议：从 Stage B1 开始，每个测试阶段建立独立的 Application；硬件结构发生实质变化时，也建立独立的 Platform。

这样工程结构最清晰，也适合我们接下来整理 GitHub 仓库、测试报告和竞赛复现材料。

你

如何给旧的application重命名?

ChatGPT

在 **Vitis 2024.2 Unified IDE** 中，我建议使用 **Clone Component（克隆组件）** 来实现 Application 的重命名。

AMD 官方文档列出了 Clone Component 功能，但没有提供直接修改 Application 名称的标准 Rename 操作。
UG1702

假设你想把原来的 `difi_dma_test` 改名为 `difi_stage_a_test`，可以这样操作。

方法一：Clone Component（推荐）

1. 在 Vitis 左侧的 **Vitis Component Explorer** 中，找到旧 Application：`difi_dma_test`。
2. 右键点击它，选择 **Clone Component。**
3. 在弹出的窗口中输入新名称：`difi_stage_a_test`。
4. 点击 OK，等待克隆完成。

完成后应该看到：

```
纯文本
Vitis Workspace
├── difi_platform_stage_a
├── difi_dma_test            ← 原组件
└── difi_stage_a_test        ← 新克隆的组件
```

检查新组件的源代码、Platform 引用以及 CMake 配置，然后执行一次 Build，确认能够成功编译。

注意：克隆不会自动删除旧组件。

新组件测试正常后，你可以右键 `difi_dma_test`，选择 Delete。删除时注意是否同时删除磁盘文件，避免误删尚未备份的源码。

方法二：新建 Application

如果右键菜单没有 Clone Component，就通过 `File → New Component → Application` 创建 `difi_stage_a_test`，再导入旧 Application 的 C 源文件。

这种方法比手动修改 Workspace 文件夹名称更可靠。

对你当前项目的建议

你现在已经完成 Stage A，正在开发 Stage B1。我建议最终组织为：

```
纯文本
vitis_ws/
├── difi_platform_stage_a/
├── difi_stage_a_test/
├── difi_platform_stage_b1/
└── difi_stage_b1_test/
```

每个测试 Application 对应自己的硬件 Platform，确保 Stage B1 更新 XSA 时不会破坏 Stage A 的可复现环境。

另外，Application 名称和 C 源文件名称是独立的。 例如 `difi_stage_a_test` 组件内部仍然可以保留 `difi_hello_world.c`，只要 CMake 正确引用该源文件即可。

你



为什么我克隆的application通过编译了，但run是灰色的，无法点击

ChatGPT

从你的截图来看，**Build 已经成功，但 Run 和 Debug 都是灰色的。**这种情况在 Vitis 2024.2 Unified IDE 中，最可能的原因是：

克隆 Application 后，新组件没有有效的 Launch Configuration（运行配置）。

Vitis 的编译配置和运行配置是独立的。Build 成功只说明已经生成 ELF，不代表 Vitis 知道应该把这个 ELF 下载到哪颗处理器、使用哪个 XSA，以及如何连接开发板。

AMD 官方 UG1400 也明确要求，为 Application 配置 Launch Configuration 后才能运行或调试。

UG1400

一、解决方法：重新创建运行配置

第一步：打开 Launch Configuration

1. 在 Flow Navigator 的 Component 下拉框中选择 `difi_stage_a_test`。
2. 把鼠标移动到灰色的 `Run` 或 `Debug` 右侧。
3. 应该会出现一个隐藏的设置图标 ⚙（Open Settings）。
4. 点击它，打开 `launch.json` 配置界面。

也可以在左侧 Component Explorer 中展开 `difi_stage_a_test → Settings`，寻找 `launch.json`。

第二步：创建新的配置

如果 `launch.json` 中没有有效配置：

1. 点击 `+ New Launch Configuration`。
2. 选择 **Embedded Standalone Application**。
3. 选择 **Configure the device and start debug**。
4. 选择对应的硬件 XSA，点击 Submit。

之后核对以下参数：

参数	应选择
Application	`difi_stage_a_test`
Processor	`ps7_cortexa9_0`
ELF	克隆后的 `difi_stage_a_test.elf`
Target Connection	Local / JTAG
XSA	Stage A 对应的 XSA
Program FPGA	启用，使用对应 Bitstream
PS Initialization	启用，使用平台生成的初始化文件

具体字段名称可能略有区别，以实际配置界面为准。

注意： 你现在运行的是 Stage A，所以应该使用 Stage A 的硬件平台和 Bitstream，而不是新的 Stage B1 XSA。

第三步：保存并运行

保存 Launch Configuration，确认其中引用的 ELF 文件确实存在。

然后返回 Flow Navigator，查看 Run / Debug 是否恢复可用。也可以直接从 Launch Configuration 界面点击 Run。

二、如果仍然是灰色的

重点检查克隆后的 Application 是否保留了旧组件的运行路径。

例如，配置中可能仍然指向：

```
纯文本
difi_dma_test/build/difi_dma_test.elf
```

而克隆后实际生成的文件可能是：

纯文本

```
difi_stage_a_test/build/difi_stage_a_test.elf
```

如果存在这种情况，需要修正 ELF 路径，或者删除旧的 Launch Configuration 后重新创建。

不要因为 Run 灰色就重新安装 Vitis、删除工程或者修改 C 源代码。

建议你现在先把鼠标悬停在灰色的 Run 或 Debug 右侧，点击出现的设置图标，打开 `launch.json`。

如果你把这个界面截图发给我，我可以直接告诉你哪些参数应该怎么填写。

你

```
==========================================
 DIFI Stage-B1: TX + AXI4-Lite + AXI DMA
 Version(44B) -> Context(108B) -> Data(1052B)
==========================================
[INFO] DIFI AXI4-Lite base=0x40000000
[INIT] TX DDR=0x0010C880 RX DDR=0x0010CC80
[CFG] Initial STATUS=0x00000012
[OK] Active Data SID=0x10203040
[OK] Active Version SID=0x55667788
[OK] Active Samples=0x00000100
[OK] Active Sample Rate=0x001E8480
[OK] Active Bandwidth=0x0016E360
[OK] Active RF Low=0x3689CAC0
[OK] Active RF High=0x00000000
[PASS] AXI4-Lite Shadow->Active commit
[START] Version S2MM armed (44 B)
[START] Engine TX enabled
[OK] Version received: Word0=0x5960000B SID=0x55667788
[VERSION] Build info=0x34FC0400
[PASS] Version Packet: 11 words
[START] Context S2MM armed (108 B)
[OK] Context received: Word0=0x4960001B SID=0x10203040
[PASS] Context Packet: 27 words, BW/RF/Fs/DPF match
[START] Data S2MM armed (1052 B)
[START] MM2S armed (1024 B IQ)
[OK] Data received: Word0=0x18600107 SID=0x10203040
[PASS] Data Packet: 7 header + 256 IQ samples
[CFG] Final STATUS=0x00000018 (Context pending should clear)
[DMA] MM2S_DMASR=0x00001002 S2MM_DMASR=0x00001002

[DIFI STAGE-B1 TEST PASSED]
测试通过了
...
```

显示更多

ChatGPT

Stage B1 正式通过！ 这意味着我们已经在真实的 Zynq-7020 硬件上完成了完整 DIFI Engine 的 **AXI4-Lite 配置、TX Scheduler 调度，以及 Data、Context、Version 三类 Packet 的生成与接收验证。**

比 Stage A 的单模块测试更进一步，这次已经验证了多个自研 RTL 模块之间的实际协同工作。

一、Stage B1 测试结果分析

DIFI Stage B1 — Board Test ✓ PASS

44 B	108 B	1052 B
Version	Context	Data
11 Words	27 Words	263 Words

- ✓ AXI4-Lite Shadow → Active Commit 通过
- ✓ TX Scheduler 首次三包调度正确
- ✓ Context 的 Fs / BW / RF / DPF 字段通过校验
- ✓ Data Packet 的 256 个 IQ 样本全部一致

还有两个细节值得关注。

1. Context Change Pending 被正确清除。

初始状态：

```
纯文本
STATUS = 0x00000012
```

最终状态：

```
纯文本
STATUS = 0x00000018
```

根据我们定义的状态寄存器：

- bit 4 = 1：Time Source 有效。
- bit 3 = 1：TX Enable。
- bit 1 = 0：Context Change Pending 已清除。
- bit 2 = 0：当前 TX 不忙。

这表明 Context 包完成发送后，`context_change_done` 与 `cfg_regs` 之间的确认机制正常工作。

2. AXI DMA 两个通道都成功完成了传输。

```
纯文本
MM2S_DMASR = 0x00001002
S2MM_DMASR = 0x00001002
```

两路 DMA 都处于 Idle 状态，且 IOC 完成标志已置位，没有显示 DMA 错误位。

二、目前的项目进度

阶段	验证对象
Stage A	Signal Data Packetizer 板级测试
Stage B1	完整 Engine TX + AXI4-Lite
Stage B2	RX Dispatch + 三类 Parser
Stage B3	FPGA TX → RX 完整双向闭环

Stage C	PC UDP + 外部 DIFI 标准验证

这里也需要明确：**Stage B1 通过代表当前测试向量和配置下的硬件功能正确，不等于完成了完整 DIFI 1.1 标准符合性认证**，也还没有验证连续高吞吐和长期稳定性。

三、下一步建议：Stage B2

我建议接下来开始 **完整 DIFI Engine 的 RX 接收解析板级验证。**

这一步与 B1 的数据流正好相反：



B2 我建议先只测试合法的 Signal Data Packet：PS 发送 1052 Bytes 的完整 DIFI Packet，PL 解析后输出 1024 Bytes IQ，同时验证 Metadata。然后再加入 Context、Version 与异常包测试。

这里还需要专门解决 **Metadata 如何让 PS 读取**的问题，因为现有 DIFI Engine 的 Metadata 使用 `valid/ready` 握手输出，并不是 AXI4-Lite 寄存器。我们会在板级适配层增加必要的状态捕获机制，避免为了上板而修改已经通过仿真的核心 Parser。

四、先把本次成果保存下来

建议保存以下文件：

```
纯文本
board/zynq7020/tests/
├── stage_a_dma_pass.log
└── stage_b1_tx_pass.log

board/zynq7020/export/
├── difi_zynq_stage_a.xsa
└── difi_zynq_stage_b1.xsa
```

对应的 `.bit`、Vitis C 源码和 Implementation 时序/资源报告也建议保留。

我的建议是：Stage B1 到这里可以验收，不必继续纠缠 TX 的细节。下一步直接开展 Stage B2，并尽可能复用现有 Zynq、AXI DMA 和 Vitis 工程。 我们已经具备继续验证 RX 方向的硬件基础。

你

好的，开始下一步

Stage B2：DIFI RX Parser 板级测试

Stage A、Stage B1 已经通过，接下来我们验证完整 DIFI Engine 的**接收解析方向**，仍然使用 Zynq-7020、Vivado/Vitis 2024.2 和现有 AXI DMA。

这次的测试链路与 Stage B1 相反：



本阶段的验收内容如下：

测试	PS 发送	期望结果
Data RX	1052 B	恢复 1024 B IQ，并获得 Data Metadata
Context RX	108 B	提交 Sample Rate、Bandwidth、RF 等 Metadata
Version RX	44 B	提交版本信息 Metadata
非法 Packet	故意破坏协议字段	报告错误，不输出非法 IQ

这里有一个重要的硬件差异：**Context 和 Version Parser 不输出 IQ，只输出 Metadata。** 因此它们不能像 Data Parser 那样直接用 DMA S2MM 接收。我们需要在板级 Wrapper 中增加一个小型调试寄存器监视器，供 PS 读取解析结果，但不修改已完成的核心 RTL。

二、Stage B2 的工程文件已经准备好

下载 Stage B2 完整工程包 ↗
包含板级 Wrapper、Vitis C 测试程序和集成说明

- difi_engine_board_b2.v ↗
- difi_stage_b2_test.c ↗
- README_STAGE_B2.md ↗

我核对了新 Wrapper 对 `difi_engine_top` 的全部 80 个端口连接，并对 C 程序进行了基础语法检查。**但这些文件尚未经过你的 Vivado 综合和 Zynq 实机验证**，因此接下来仍需要按步骤检查。

三、修改 Vivado 工程

建议复制 Stage B1 工程，建立 `difi_zynq_stage_b2`，不要覆盖已成功的 B1 工程。

第一步：替换板级 Wrapper

添加 `difi_engine_board_b2.v` 到 Design Sources，并保留完整的 DIFI Engine 九模块 RTL。

打开 Block Design，删除原来的 `difi_engine_board_b1_0` 实例，通过 `Add Module` 添加新的 `difi_engine_board_b2`。

第二步：调整 DMA 数据通路

AXI DMA	新 Wrapper
`M_AXIS_MM2S`	`S_AXIS_PKT`
`S_AXIS_S2MM`	`M_AXIS_IQ`

AXI4-Lite 的 `S_AXI_CTRL`、时钟和复位继续沿用 B1 的连接方法。

注意：B2 的数据方向与 B1 相反。 现在 MM2S 搬运的是完整 DIFI Packet，而 S2MM 接收的是解析后的 IQ 数据。

第三步：修改现有 AXI GPIO

为了让 ARM 读取 Metadata，我们复用之前的 `axi_gpio_0`，不需要再新增一个 AXI4-Lite Slave。

双击 AXI GPIO，将它改为：

参数	设置
Enable Dual Channel	勾选
GPIO Channel 1 Width	5
GPIO Channel 1 Direction	All Outputs
GPIO Channel 2 Width	32
GPIO Channel 2 Direction	All Inputs

然后连接：

- `gpio_io_o[4:0]` → `dbg_page[4:0]`
- `dbg_rdata[31:0]` → `gpio2_io_i[31:0]`

其中 `dbg_page` 是 PS 写入的寄存器选择编号，`dbg_rdata` 是当前选中监视寄存器的值。

我们可以读取三类 Parser 的 Metadata、成功计数和错误计数，而不需要修改核心协议 RTL。

四、重新生成硬件并进行测试

完成连接后：

1. `Validate Design`
2. 确认 Address Editor 中 DIFI、DMA 和 GPIO 地址未发生意外变化。
3. 生成 HDL Wrapper（或更新已有 Wrapper）。
4. 重新综合、Implementation，确认时序收敛。
5. 生成 Bitstream，导出 `difi_zynq_stage_b2.xsa`。

在 Vitis 中，建议新建 `difi_platform_stage_b2` 和 `difi_stage_b2_test`，分别关联新的 XSA 和 C 程序。

测试程序会先验证 GPIO 监视器，再发送 Version、Context、Data，最后测试非法 Packet Type 是否被 RX Dispatch 拒绝。

正常情况下，最终应得到：

```
纯文本
[OK] B2 metadata monitor detected and page selector works
[OK] RX Stream ID filtering enabled
[PASS] Version Parser: metadata and info verified
[PASS] Context Parser: metadata BW/RF/Fs verified
```

```
[PASS] Data Parser: all 256 IQ words and metadata match
[PASS] Unsupported type rejected by RX Dispatch

[DIFI STAGE-B2 TEST PASSED]
```

我们现在先完成 Vivado 的 Stage B2 Block Design 修改。 这次改动主要集中在 DMA 连接方向和 AXI GPIO 双通道配置，PS、DDR、时钟复位及 DIFI 核心都可以复用。

完成 `Validate Design` 后，把新的 Block Design 截图发给我，我们先核对连接，再进入综合和真实板级测试。

difi_stage_b2_test.c
代码

README_STAGE_B2.md
文档

你

==========
DIFI Stage B2: Full Engine RX + DMA
==========
[INFO] DIFI CTRL base = 0x40000000
[OK] B2 metadata monitor detected and page selector works
[OK] RX Stream ID filtering enabled
[START] Version MM2S: 44 B
[PASS] Version Parser: metadata and info verified
[START] Context MM2S: 108 B
[PASS] Context Parser: metadata BW/RF/Fs verified
[START] IQ S2MM: 1024 B
[START] Data MM2S: 1052 B
[PASS] Data Parser: all 256 IQ words and metadata match
[START] InvalidType MM2S: 32 B
[PASS] Unsupported type rejected by RX Dispatch
[DMA] MM2S=0x00001002 S2MM=0x00001002
[MON] data=1 ctx=1 ver=1 errors=1/0/0/0 last=0x00000101

[DIFI STAGE-B2 TEST PASSED]

这算是测试通过了吗
...
显示更多

ChatGPT

是的，Stage B2 已经通过！ 从你提供的完整串口日志来看，所有预定测试项均成功执行，DMA 状态正常，Metadata 和 IQ 数据校验也全部通过。

这意味着我们已经在真实 Zynq-7020 上分别验证了完整 DIFI Engine 的 **TX 发送方向（Stage B1）和 RX 接收解析方向（Stage B2）**。

一、Stage B2 测试结果

DIFI Stage B2 — Hardware Verification ✓ PASS

正常 Packet 解析	IQ 样本正确率
3/3	**256/256**

异常包检测	DMA 传输
1/1	**正常**

测试项目	日志结果
AXI GPIO Metadata Monitor	`[OK]`
RX Stream ID 过滤配置	`[OK]`
Version Parser	`[PASS]`
Context Parser	`[PASS]`
Data Parser	`[PASS]`
非法 Packet Type 检测	`[PASS]`

尤其值得注意的是最后两行：

```
纯文本
[DMA] MM2S=0x00001002 S2MM=0x00001002
[MON] data=1 ctx=1 ver=1 errors=1/0/0/0 last=0x00000101
```

DMA 的 `0x00001002` 表示 Idle 和 IOC 状态位置位，没有 DMA 错误标志。

Metadata Monitor 进一步证明：

- `data=1`：成功提交一个 Data Packet。
- `ctx=1`：成功提交一个 Context Packet。
- `ver=1`：成功提交一个 Version Packet。
- `errors=1/0/0/0`：RX Dispatch 检测到一次错误，其余三个 Parser 未报告错误。

由于你恰好发送了一个非法 Packet Type，所以 **一次 Dispatch 错误正是预期结果**，不是测试失败。

二、目前项目的进度

- Stage A — 已通过
  单独 Data Packetizer 板级验证
- Stage B1 — 已通过
  完整 Engine TX + Scheduler + AXI4-Lite
- Stage B2 — 已通过
  完整 Engine RX + Dispatch + 三类 Parser
- Stage B3 — 待开展
  TX → RX 板级端到端闭环
- Stage C — 待开展
  PC UDP 通信与外部 DIFI 验证

三、Stage B2 通过意味着什么?

我们可以确认当前测试覆盖范围内的几个事实：

第一，完整 RX 数据通路在真实硬件上可用。 PS 经 AXI DMA 发送 Packet，RX Dispatch 能够正确分流，三个 Parser 可以解析各自负责的数据。

第二，Data Parser 的整包缓存及 IQ 输出机制可用。 256 个 IQ Sample 均被正确恢复，Metadata 也与测试预期一致。

第三，基本异常包检测机制可用。 非法 Packet Type 被 Dispatch 检测到，没有造成其他 Parser 报错。

但是，这还不等于 RX 的全部边界情况已经通过。比如 TLAST 提前结束、Stream ID 不匹配、错误 Sequence、恶意构造的 Packet Size，以及持续高吞吐背压，目前这份日志还没有全部覆盖。

四、下一步应该做什么？

我建议直接开展 **Stage B3：完整 TX → RX 板级闭环验证。**

目标数据流为：

```
PS IQ → DMA → FPGA TX Packetizer → DIFI Packet → FPGA RX Parser → DMA → PS IQ
```

其中还应验证 Version、Context 和 Data 三类包经过内部通路后，Context/Version Metadata 正确提交、Data IQ 完全一致。

和 Stage B1、B2 分别测试不同的是，B3 要证明**完整协议引擎的发送端与接收端能够直接协同工作**，而不仅仅是各自能够处理 PS 提供的数据。

这一步建议优先采用 PL 内部 AXI4-Stream 回环，暂时不引入 UDP，降低调试复杂度。

另外，请将当前日志保存为：

```
board/zynq7020/tests/stage_b2_rx_pass.log
```

并保留 Stage B2 的 XSA、Bitstream、Vitis 源码及实现报告。

结论：Stage B2 可以正式验收，我们不必再反复验证相同的单包测试向量。下一步进入 Stage B3，完成完整 DIFI Engine 的第一次真实 FPGA 端到端闭环。

你

好的，开始下一步

ChatGPT

Stage B3 的工程文件已经准备好了。我沿用了你已经通过板级验证的 B2 Wrapper，并完成了与当前 GitHub `difi_engine_top` **全部 80 个端口**的核对。Vitis 测试代码也通过了基础 C 语法检查。

一、下载 Stage B3 工程

下载 Stage B3 完整工程包 ↗
包含 Verilog Wrapper、Vitis C 测试程序和集成说明

difi_engine_board_b3.v ↗

difi_stage_b3_test.c ↗

README_STAGE_B3.md ↗

这些文件尚未经过你的 Vivado 综合和 Zynq 实机测试，所以我们仍按上一阶段的方法逐步验证。

二、修改 Vivado Block Design

先备份 Stage B2 工程，再建立 `difi_zynq_stage_b3`。

这次最大的好处是：**不用新增任何 IP。**

1. 将 `difi_engine_board_b3.v` 添加到 Design Sources，保留已有九模块 RTL 和 `difi_engine_top.v`。
2. 在 Block Design 中删除 `difi_engine_board_b2_0`，通过 `Add Module` 添加 `difi_engine_board_b3`。
3. 重新连接 DMA 的 AXI4-Stream 接口：

AXI DMA	Stage B3 Wrapper

`M_AXIS_MM2S`	`S_AXIS_IQ`
`S_AXIS_S2MM`	`M_AXIS_IQ`

4. 将 `S_AXI_CTRL` 继续接 PS GP0 对应的 SmartConnect。
5. 时钟接 `FCLK_CLK0`，复位接 `peripheral_aresetn`。
6. AXI GPIO 保留 B2 的双通道配置：CH1 为 5-bit 输出接 `dbg_page`，CH2 为 32-bit 输入接 `dbg_rdata`。

内部 DIFI TX → RX 的 AXI4-Stream 连接已经写在新的 Wrapper 中，不需要在 Block Design 里再连接一次。

三、Stage B3 比 B2 增加了什么验证?

除了 B2 已有的 Metadata 监视器，我还增加了两个 GPIO 调试页：

调试页	内容
Page 29	TX 已完整发送的 Packet 数
Page 30	TX 实际握手成功的 Word 总数

通过它们可以核对发送的报文是否全部进入内部接收链路。

这次我们发送 **两组不同的 256 个 IQ Sample**，所以最终预期结果为：

完整 DIFI Packet	DIFI Word 总数	IQ 样本比较
4	**564**	**512/512**

4 个 Packet = Version ×1 + Context ×1 + Data ×2；Data Sequence 应依次为 0、1，所有 RX 错误计数器应为 0。

四、生成 XSA 后的 Vitis 操作

完成 `Validate Design`、Synthesis、Implementation、Bitstream 后，导出：

`difi_zynq_stage_b3.xsa`

然后在 Vitis 2024.2 中分别建立：

- Platform：`difi_platform_stage_b3`
- Application：`difi_stage_b3_test`

Application 的 `src` 目录加入下载的 `difi_stage_b3_test.c`，确保 CMake 引用正确文件名。

测试程序将自动提交 AXI4-Lite 配置、等待初始 Version/Context 经 RX 解析成功、发送两组 IQ，并检查 DMA、Metadata、Sequence 和全部样本。

我们先完成 Vivado 的 B3 Block Design 修改，并运行 Validate Design。

成功后把 Block Design 截图发给我，我会检查连接，再进入实际板级运行。这样能够把硬件连接问题和 Vitis 软件问题分开排查。

difi_stage_b3_test.c
代码

README_STAGE_B3.md
文档

你

```
==========================================
 DIFI Stage B3: FPGA TX -> RX Loopback
==========================================
[INFO] DIFI AXI-Lite base=0x40000000
```

[OK] Stage B3 monitor and reset counters
[PASS] AXI4-Lite configuration/commit + RX SID filters
[START] Engine TX enabled; waiting for Version -> Context
[PASS] TX->RX Version + Context metadata, 2 packets / 38 words
[START] Data #0: S2MM armed (1024 B)
[START] Data #0: MM2S started (1024 B)
[PASS] Data #0 loopback: 256/256 IQ match, seq=0
[START] Data #1: S2MM armed (1024 B)
[START] Data #1: MM2S started (1024 B)
[PASS] Data #1 loopback: 256/256 IQ match, seq=1
[DMA] MM2S=0x00001002 S2MM=0x00001002
[MON] TX packets=4 beats=564 | RX data=2 ctx=1 ver=1 | errors=0/0/0/0
[MON] Last error=0x00000000, STATUS=0x00000018

[DIFI STAGE-B3 TEST PASSED]测试通过了
...

显示更多

ChatGPT

Stage B3 正式通过！至此，我们已经在真实 Zynq-7020 FPGA 上完成了 DIFI 协议引擎的 TX、RX 以及端到端硬件闭环验证。

从这次日志来看，结果不仅是程序输出了 PASS，而且各项计数器之间也完全一致，没有发现明显异常。

一、Stage B3 测试结果分析

DIFI Stage B3 — FPGA TX → RX Loopback PASS

完整 Packet 数	AXIS 握手 Word 数
4 / 4	**564**
IQ 正确样本	RX 错误事件
512 / 512	**0**

1. TX 和 RX 的 Packet 数完全匹配

Packet	TX 发送数	RX 解析数
Version	1	1
Context	1	1
Data	2	2
合计	4	4

这与日志中的 `TX packets=4 beats=564` 完全一致。

2. Sequence 和 IQ 均正确

纯文本

```
[PASS] Data #0 loopback: 256/256 IQ match, seq=0
[PASS] Data #1 loopback: 256/256 IQ match, seq=1
```

说明两次独立 Data Packet 发送后，接收端正确恢复了全部 512 个 IQ Sample，且 Packet Sequence 连续递增。

3. DMA 和错误计数正常

```
纯文本
MM2S=0x00001002 S2MM=0x00001002
errors=0/0/0/0
Last error=0x00000000
```

两个 DMA 通道完成传输，没有报告 DMA 错误；四路 RX 错误事件计数均为零。

因此，在当前测试条件下，我们可以认为 **PL 内部 DIFI TX → RX 闭环已经通过硬件功能验证。**

二、目前完成了多少?

阶段	工作内容
RTL	九模块协议引擎、系统仿真
Stage A	Data Packetizer 单模块上板
Stage B1	三类 Packet TX + AXI4-Lite
Stage B2	三类 Packet RX + 错误检测
Stage B3	完整 Engine TX → RX 回环
Stage C	UDP 网络互通、外部协议验证
Stage D	性能、可靠性、竞赛交付

这里有一个重要区别：**我们现在已经证明 FPGA 引擎在测试范围内能够正确工作，但还没有证明它能够与外部 DIFI 系统互操作，也没有实测持续吞吐率与延迟。**

这些将是之后工作的重点。

三、下一步建议：Stage C，PC ↔ Zynq UDP 通信

按照之前冻结的项目规划，下一阶段应该接入以太网。

我建议使用 Zynq PS 自带的 Gigabit Ethernet MAC（GEM）和 lwIP，暂时不在 PL 内实现 UDP/IP 协议栈。



为了控制工作量，我建议分成三个小步骤：

1. **Stage C1：** 先让 PC 和 Zynq PS 实现 UDP 双向通信，不接 DIFI Engine，确认网络环境可靠。
2. **Stage C2：** 接入 DIFI Engine，实现 PC 发送 DIFI Packet → Zynq FPGA 解析，以及 FPGA 生成 DIFI Packet → PC 接收。
3. **Stage C3：** 使用 Wireshark、DIFI Consortium 的参考工具及 PCAP 测试文件，进行独立协议验证。

其中 C2 要特别注意每个 UDP Datagram 对应一个完整 DIFI Packet，不能丢失 `TLAST` 所代表的包边界。

四、现在应该保存什么？

建议把 Stage B3 作为一个独立的已验证版本保存。

至少保留 `stage_b3_loopback_pass.log`、B3 的 Bitstream/XSA、Vitis 测试源码，以及 Implementation 的资源和时序报告。

我的建议是：现在冻结 Stage B3 的硬件版本，不再为已经通过的基本 TX/RX 功能增加新模块。下一步直接开展 Stage C1 的 Zynq PS 千兆以太网与 PC UDP 通信测试。

这样我们能够在已有可靠 FPGA 数据通路的基础上，进一步完成面向外部系统的真实通信闭环。

你

但是，有些外部模块，比如说FIFO、定时触发的脉冲生成器等模块还没做，客观上来说，我们到底先干什么？

ChatGPT

你这个提醒非常重要。重新对照我们之前制定的《DIFI 1.1 协议实现规格》、RTL 接口文档，以及刚刚完成的 Stage B1/B2/B3 上板测试后，我认为**之前建议立即进入 UDP 通信，确实有些跳跃了。**

目前应该先补齐部分系统级功能，再进行外部网络集成。但也不能简单地认为"所有外围模块都必须先写完"。

一、首先明确：我们现在完成的是什么？

我们目前完成的是**DIFI 协议核心引擎的功能验证**，而不是完整的、能够自主持续运行的通信系统。

功能	当前情况
Data / Context / Version 生成与解析	RTL、板级测试均通过
TX Scheduler、RX Dispatch	已完成并通过板级闭环
AXI4-Lite 配置与 Commit	已通过
周期性 Context / Version 发送	Scheduler 支持，但外部触发器还没有
Time Source	目前使用固定时间戳
FIFO / 弹性缓冲	有 Parser 内部 BRAM Buffer，但缺少系统级输入输出 FIFO
错误与状态监测	B2/B3 有 GPIO 调试计数器
持续数据流测试	目前主要是单包、两包 DMA 测试
UDP 网络收发	尚未接入
外部 DIFI 标准验证	尚未完成

特别要澄清：**没有额外 FIFO 并不代表当前架构错误。**

B3 已经证明，同一 100 MHz 时钟域下，AXI DMA 与协议引擎可以通过 `TVALID/TREADY` 正确传输数据。对于能够接受背压的数据源，FIFO 主要改善缓冲和吞吐表现，并不是建立正确数据通路的前提。

但是，目前 `context_periodic_trigger` 和 `version_periodic_trigger` 始终连接 `1'b0`，使得已经实现的自动周期调度功能根本没有在真实硬件上运行过。这就是明确的功能缺口。

因此，我建议下一阶段先做 Stage B4：系统定时与自主调度，然后再开展连续流/FIFO 集成测试。

二、我建议采用以下实施顺序

顺序	阶段	具体任务
1	B4：定时与自主调度	周期脉冲生成器、Time Source、自动 Context/Version 发送
2	B5：连续数据流与缓冲	连续多包测试、背压测试、按需插入 FIFO、异常恢复

3	C1：PS 网络通信	Zynq GEM + lwIP，PC UDP 双向通信
4	C2：网络与 DIFI Engine 集成	PC ↔ UDP ↔ PS ↔ DMA ↔ DIFI Engine
5	C3：外部协议验证	DIFI Consortium 工具、Wireshark、PCAP
6	D：性能与竞赛交付	吞吐、时延、Fmax、资源、稳定性、报告
7	扩展优化	更多缓冲、复杂时间同步、控制协议等

其中 B5 不意味着必须设计一套复杂 FIFO 系统，而是必须完成连续数据传输验证，并根据实际瓶颈决定 FIFO 的位置和深度。

三、B4 具体做什么?

我建议先实现两个外围 RTL 模块。

`difi_time_source.v`

由 PS 设置 UTC 时间基准，PL 根据 100 MHz 时钟推进时间，提供 `time_sec`、`time_ps`、`time_valid`。

↓

`difi_periodic_trigger_gen.v`

读取 Engine 已有的 Context/Version 周期配置，生成两个独立的单周期触发脉冲。

↓

现有 `difi_engine_top`

不修改九模块协议核心

TX Scheduler → Data / Context / Version

先写周期脉冲生成器，再写 Time Source。

因为现有 Engine 已经提供:

```verilog
output wire [31:0] cfg_context_period_ms;
output wire [31:0] cfg_version_period_ms;

input wire context_periodic_trigger;
input wire version_periodic_trigger;
```

这意味着调度器的接口已经准备好了，只差外部模块接入。100 MHz 下每毫秒对应 100,000 个时钟周期，可以先产生统一的毫秒 Tick，再用两个周期计数器独立触发。

Time Source 则先采用 PS 设置初始 UTC 时间、PL 本地递增的方案，不引入 GPS/PPS 或 NTP 硬件同步。它可以验证时间戳动态更新，但没有校准前不能宣称高精度 UTC 同步。

B4 的验收标准是：系统上电、配置并使能 TX 后，即使没有 IQ 输入，也能周期性发送 Version 和 Context；修改周期配置后新周期能够生效；之后输入 IQ，Data Packet 仍然正确发送，且时间戳会随时间推进。

四、FIFO 到底什么时候做?

我不建议现在立即编写一个通用 FIFO 模块，然后在每条 AXI4-Stream 通路上随意插入。

应该根据数据来源决定。

场景	FIFO 必要程度	我的建议
PS AXI DMA → TX IQ	非必需	当前能够正确背压，先保留直连
TX Engine → PS DMA / 网络	根据缓冲及软件调度需要	连续传输时再评估

PS DMA / UDP → RX Engine	根据突发接收情况需要	先保证完整 Packet 边界
实际 ADC → TX Engine	通常非常重要	数据源不能停顿时需要弹性缓冲与溢出处
跨时钟域数据传输	需要 CDC 机制	优先使用合适的异步 FIFO

需要特别注意的是，**FIFO 不能凭空解决持续速率不匹配。**如果输入长期快于输出，再大的有限 FIFO 最终也会满。

而且对于真实 ADC 输入，FIFO 会增加采集到打包之间的延迟。若时间戳要求表示第一个 IQ 样本的实际采集时刻，还必须考虑采集时间与缓存数据的关联，不能简单地拿 FIFO 出口时的时间代替采样时间。

所以 B5 应当先测量连续流的表现，再决定是否增加 AXI4-Stream Data FIFO、其深度及位置。如果仅仅用于短期缓冲，优先考虑 Vivado 自带的 AXI4-Stream FIFO IP，没有必要重新实现通用存储 FIFO。

五、还有一个重要的时间安排问题

今天已经是 10 月 12 日，我们此前希望 10 月底完成竞赛初版。

因此我建议：

10/13—10/15 B4：外围定时模块
定时触发、时间源、自动周期收发，上板验证

10/16—10/18 B5：连续流测试
先测试再决定 FIFO；完成必要的缓冲和错误观测

10/19—10/23 C1/C2：网络集成
PC UDP ↔ Zynq PS ↔ DIFI Engine

10/24—10/26 C3：外部 DIFI 验证
Wireshark、参考工具和 PCAP 证据

10/27—10/31 性能与交付
定量数据、文档、演示与最终回归

这个安排很紧。尤其 PS 网络与 UDP 集成存在不确定性，因此如果你和师兄可以并行开发，**建议你负责 B4/B5，他同时进行 C1 的 lwIP 网络连通测试。**这样能够降低最后集中调试网络的风险。

六、最终决定

我认为现在最合理的工程策略是：

停止扩展协议核心 → 补齐定时与时间源 → 验证连续数据流、确定 FIFO 需求 → 集成 UDP → 做外部验证与性能测试。

既不应该因为 B3 通过就直接把外围问题全部推到最后，也不应该为了项目看起来完整而盲目增加 FIFO、缓存、CDC 和其他 IP。

因此我们的下一个具体任务就是实现 `difi_periodic_trigger_gen.v`，并编写独立 Testbench。 验证正确后再接入 B3 的系统 Wrapper，形成 Stage B4。这样每一步都有明确的功能目标和可检验的硬件结果。